SCSE2123: Software Requirements Engineering

Semester 01, 2025/2026

Requirements Model Document

Phase 2 report (PR2) : Requirements Model Document (RMD)

Week 11, 19/12/2025

Prepared by: Group 9

Group name: Visionscope

NAME	MATRIC NO
LEE MING DA	A24CS0301
GAN RUI EN	A24CS0249
TAN CHOON DA	A24CS0317
Mohammed Mohsen Mohammed Al-Sakkaf	A23CS4026

DOCUMENT CONTROL

General Information

Document Name	:	ETIQA-RMD-GR9 – Requirements Model Document
Audience	:	Etiqa Stakeholder
Prepared by	:	Visionscope
Prepared Date	:	12/11/2025
Revision Date	:	19/12/2025
Printed Date	:	19/12/2025
Reference Number	:	SCSE2123-G9-RMD
Version	:	1.0
Document Level	:	Confidential

a. Current Version Document

	Name	Designation	Signature	Date
Prepared by:	Lee Ming Da	Project Manager		19/12/2025
	Tan Choon Da	Moderator		19/12/2025
	Gan Rui En	Task Manager		19/12/2025
	Mohammed Mohsen Mohammed Al-Sakkaf			
Verified by:	***Dr. Noraini rahim***	***Head of Project***	***TBD***	***TBD***

Table of Content

Item	Page No	Prepared by	Moderated by
1. Introduction	5	Tan Choon Da	Dr Radziah
2. Functional Perspective: Use Case (UC) Documentation (Diagram – UML UCD)	6-8	Lee Ming Da	Dr Radziah
2.1 Functional Perspective: Use Case (UC) Documentation (Specification – UCS) & UML Activity Diagram (AD) UC001: Login to Etiqa+ UC002 View Account Balance UC003 View Transaction History UC004 Request Payout UC005 Upload Verification Document UC006 Process Incoming Payout UC007 Withdraw Funds UC009 Generate Transfer Report UC010 Reinvest Product UC014 Update User Balance UC015 View Policy Portfolio UC016 Download Policy Certificate	9-28	Lee Ming Da Tan Choon Da Gan Rui En Mohammed	Dr Radziah

SC001 Block Attack SC002 Consolidate Payout Data SC003 Manage Verified Bank Accounts SC004 Perform Regulatory Checking MUC01 Account Takeover MUC02 Submit Fraudulent Claim MUC03 Intercept Fund Transfer MUC04 Bypass Compliance Check			
3. Data Perspective: UML Domain Class Diagram (DCD)	29-32	Tan Choon Da	Dr Radziah
4. Behavioral Perspective: State-Transition Diagram (STD)	33-36	Gan Rui En	Dr Radziah
5. Conclusion	37	Tan Choon Da	Dr Radziah
6. References	38	Gan Rui En Lee Ming Da Tan Choon Da	Dr Radziah
7. Appendices	39-41	Gan Rui En Tan Choon Da	Dr Radziah

1. INTRODUCTION

This report builds directly on the foundation we set in Phase 1 (URD). Previously, we defined the business needs for the **Etiqa Digital Payout (eMoney)** system in text format now in Phase 2 so our team is translating those written requirements into visual technical models. The primary goal here is to create a clear blueprint using the Unified Modeling Language (UML) that developers can actually use to build the system, rather than just relying on a list of wishes.

Purpose :
The purpose of this report is to analyze the system requirements through three distinct modeling perspective, as required by the project guidelines:

- **Functional perspective:** We use Use Case Diagrams (UCD) and Activity Diagrams (AD) to map out exactly how users like the Customer and Claim Officer interact with the platform step by step.
- **Data perspective:** We created a Domain Class Diagram (DCD) to structure the data we need to store, ensuring we capture details like Policy IDs and Transaction Logs correctly.
- **Behavioral perspective:** We use a State-Transition Diagram (STD) to track the precise lifecycle of complex objects. This scope is specifically how key objects change states during operations, using UML State Transition Diagram

Scope:

This document's scope includes modeling the central Etiqa eMoney ecosystem. It focuses on customer fund utilization, automated payout mechanisms, and regulatory compliance reporting.

The technical development team, system architects, and project stakeholders (the Etiqa Claims, Finance, and Compliance departments) are the target audience for this document. Before the comprehensive Software Requirements Specification (SRS) is created in Phase 3, it acts as the main source for verifying the system's logical design. It guarantees that the business rules and regulatory frameworks (RMIT) set forth in the URD are in line with all functional flows, data structures, and system behaviors.

2. FUNCTIONAL PERSPECTIVE: USE CASE DOCUMENTATION

2.1 Use Case Diagram (UCD)

The UCD for the Etiqa eMoney System illustrates the functional boundaries of the platform and the interactions between primary stakeholders and the system.

Actors identified in the UCD include

- **Customer:** The primary user who interacts with the system to request payouts, manage funds, reinvest in products, and view transaction history or policy portfolios.
- **Payout Service System:** The backend technical actor responsible for orchestrating the incoming payout process and ensuring the user's ledger balance is updated accurately.
- **Source System:** The external entity that initiates the flow of funds into the platform by triggering the process for incoming payouts.
- **Bank System:** The external financial institution actor that interfaces with the platform to facilitate fund withdrawals and manage the verification of external bank accounts.
- **Document Management System:** The support actor responsible for handling the upload, validation, and storage of verification documents required for payout requests.
- **Etiqa+ App:** It is explicitly responsible for receiving push notifications to update the user balance and serves as the display interface for viewing the policy portfolio.
- **Attacker:** The misuse case actor explicitly included to model security threats such as account takeover or fraud, highlighting the system's necessary mitigation strategies.

Relationship Justification

- <<include>>: Used for mandatory sub-processes
- <<threatens>> & <<mitigate>>: These misuse case relationships identify specific security risks and the functional use cases designed to counteract them.

Functional Use Cases

Every use case in the graphic adheres to the naming convention, in which the noun denotes the action's target or goal and the verb specifies the action carried out. This norm guarantees that system functionality is clear, consistent, and simple to comprehend.

The main functional use cases include:

- UC001: Login to Etiqa+
- UC002: View Account Balance
- UC003: View Transaction History
- UC004: Request Payout
- UC005: Upload Verification Documents
- UC006: Process Incoming Payout
- UC007: Withdraw Funds
- UC009: Generate Transfer Report
- UC010: Reinvest Product
- UC014: Update User Balance
- UC015: View Policy Portfolio
- UC016: Download Policy Certificate

Misuse Cases and Security Use Cases

Misuse cases are used to model potential security threats to the system:

- MUC01: Account Takeover threatens UC001: Login to Etiqa+
- MUC02: Submit Fraudulent Claim threatens UC004: Request Payout
- MUC03: Intercept Fund Transfer threatens UC007: Withdraw Funds
- MUC04: Bypass Compliance Check threatens UC010: Reinvest Product

Each misuse case is mitigated by a corresponding security use case:

- MUC01: Account Takeover mitigated by SC001: Update Security Settings
- MUC02: Submit Fraudulent Claim mitigated by SC002: Consolidate Payout Data
- MUC03: Intercept Fund Transfer mitigated by SC003: Manage Verified Bank Accounts
- MUC04: Bypass Compliance Check mitigated by SC004: Perform Regulatory Check

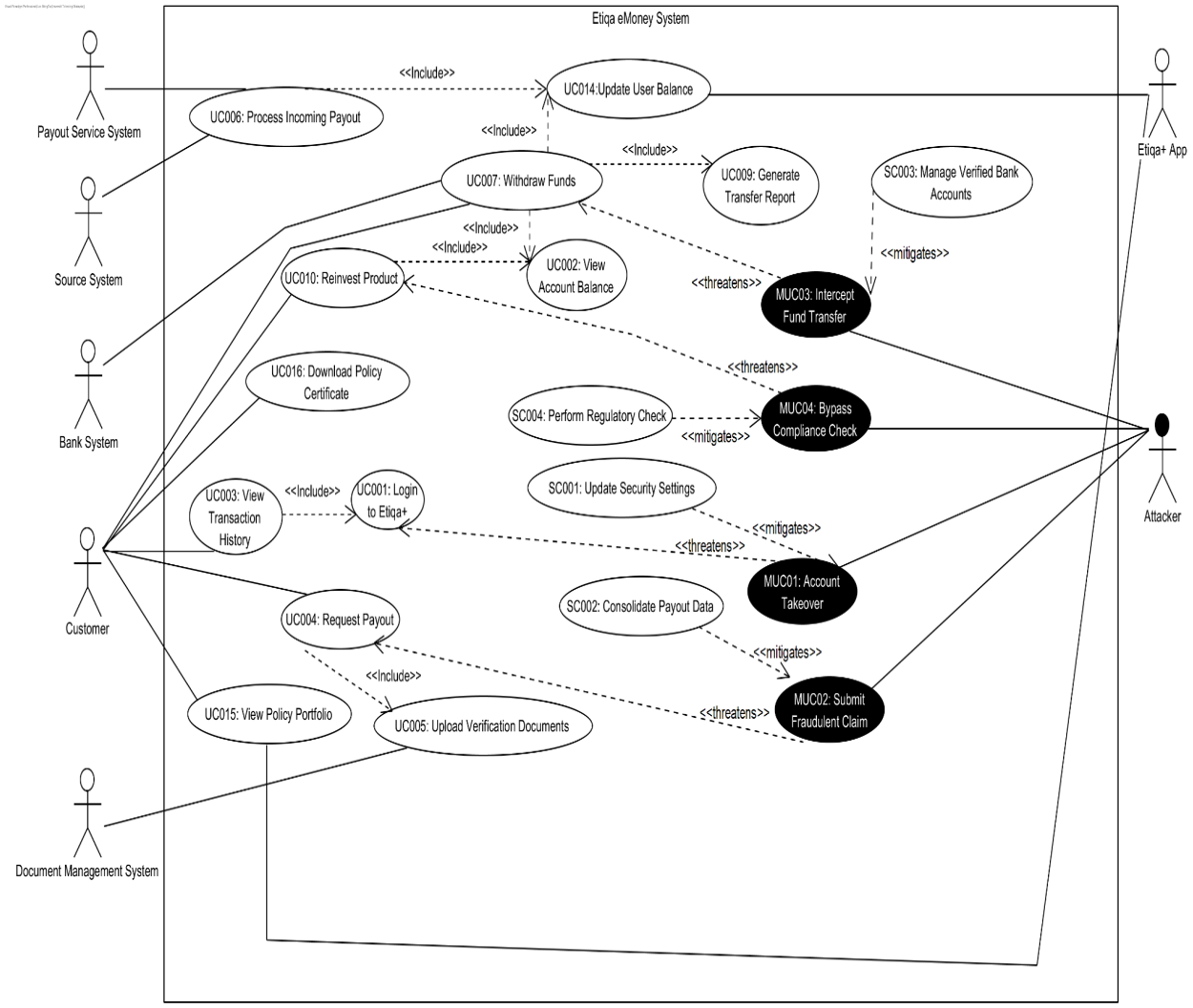


Figure 1: Use Case Diagram for Etiqa eMoney System

The Use Case Diagram for the Etiqa eMoney System illustrates the functional interactions between primary users, backend services, and potential security threats. It defines the system boundary where the Customer interacts with core features such as requesting payouts, withdrawing funds, and reinvesting products, while technical actors like the Payout Service System, Bank System, and Document Management System facilitate backend processing and verification. In addition, this diagram models an Attacker who initiates Misuse Cases such as Intercept Fund Transfer or Submit Fraudulent Claims. These threats are directly linked to mitigation strategies, such as Perform Regulatory Check and Manage Verified Bank Accounts, demonstrating how the system proactively defends against specific risks while delivering its primary financial services.

2.2 UML Activity Diagram (AD)

The following Activity Diagram models the dynamic behavior and logic flow for the Etiqa eMoney System functions.

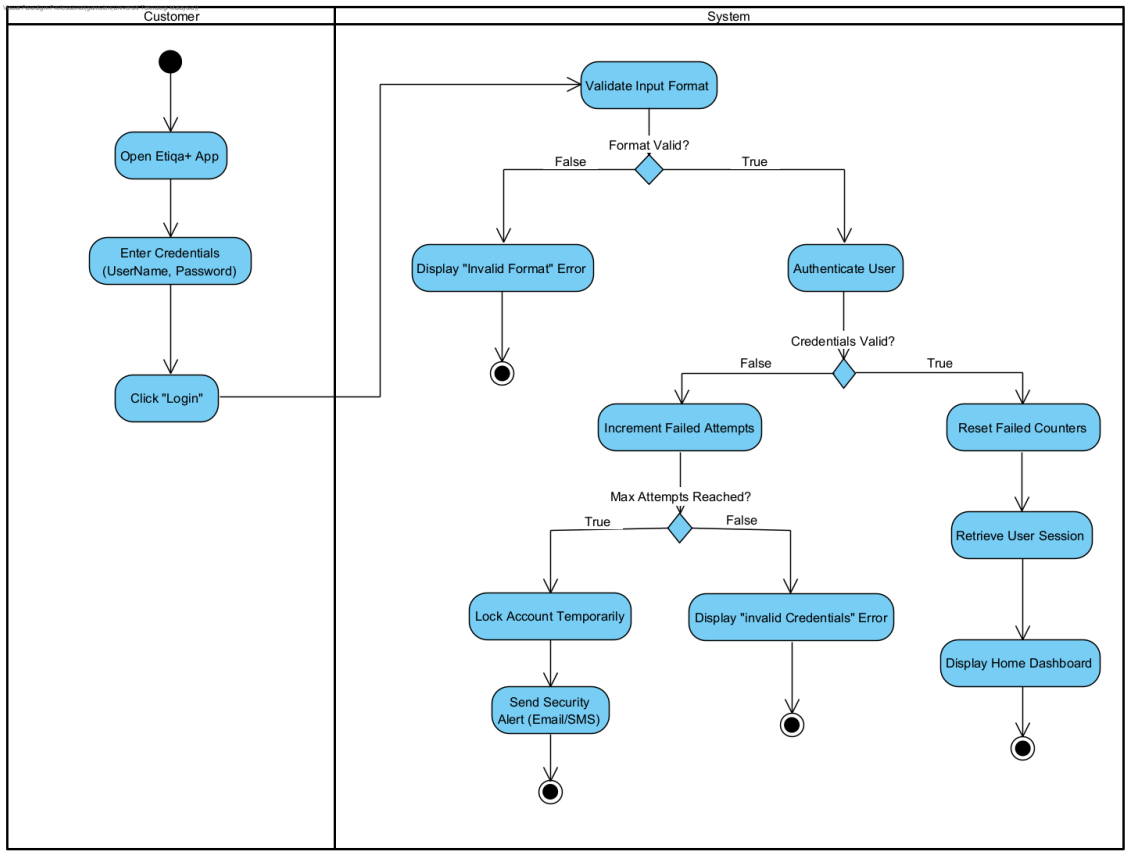


Figure 2.2.1: Activity Diagram for UC001 Login to Etiqa+

For UC001, this diagram details the primary authentication process for accessing the system. The flow begins when the Customer opens the application and inputs their credentials including username and password. The system first performs a format validation to ensure the inputs meet standard requirements. If the format is valid, the system attempts to authenticate the user against the database.

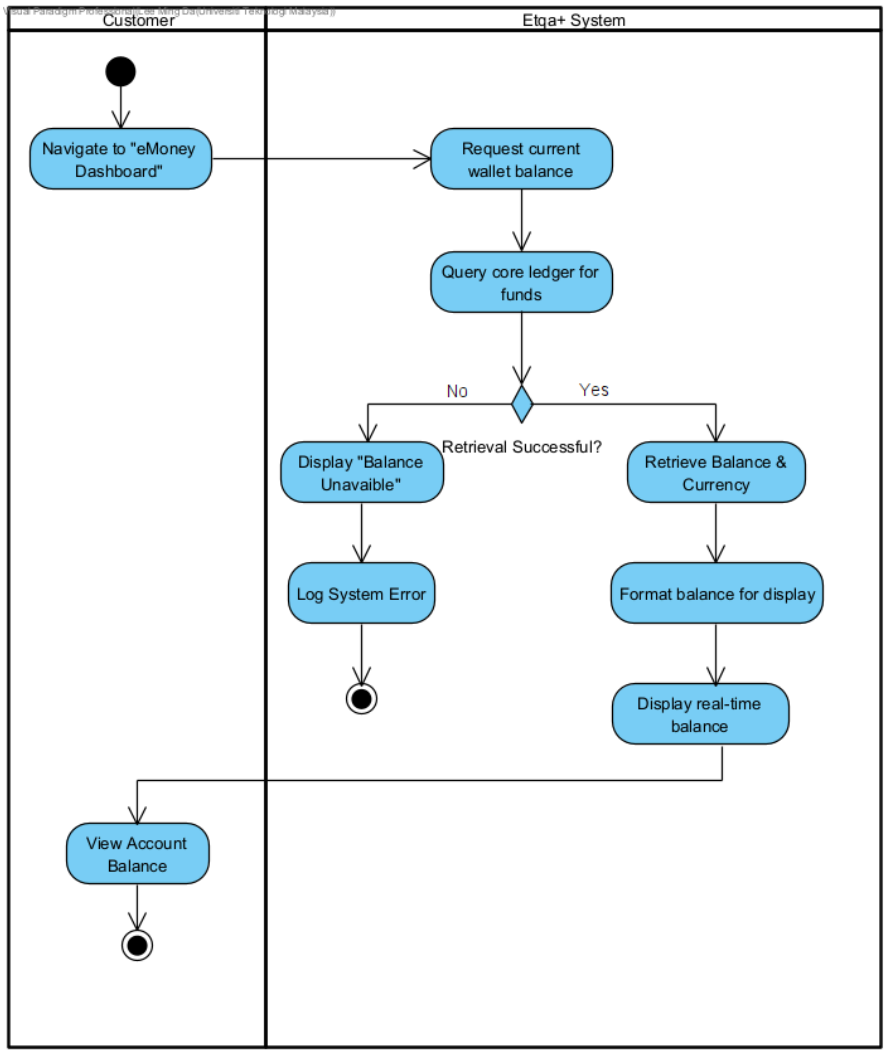


Figure 2.2.2: Activity Diagram for UC002 View Account Balance

For UC002, this use case ensures customers are never left guessing about their money. In the old system, funds could be "in transit" with no visibility, but this feature gives users a real-time, accurate view of their wallet balance directly on the dashboard. By querying Etiqa's central Core Ledger, the app shows the exact amount available the moment a claim is paid out or a purchase is made. This immediate feedback loop eliminates anxiety for the customer and significantly reduces the need for them to contact support just to check if a payment has arrived.

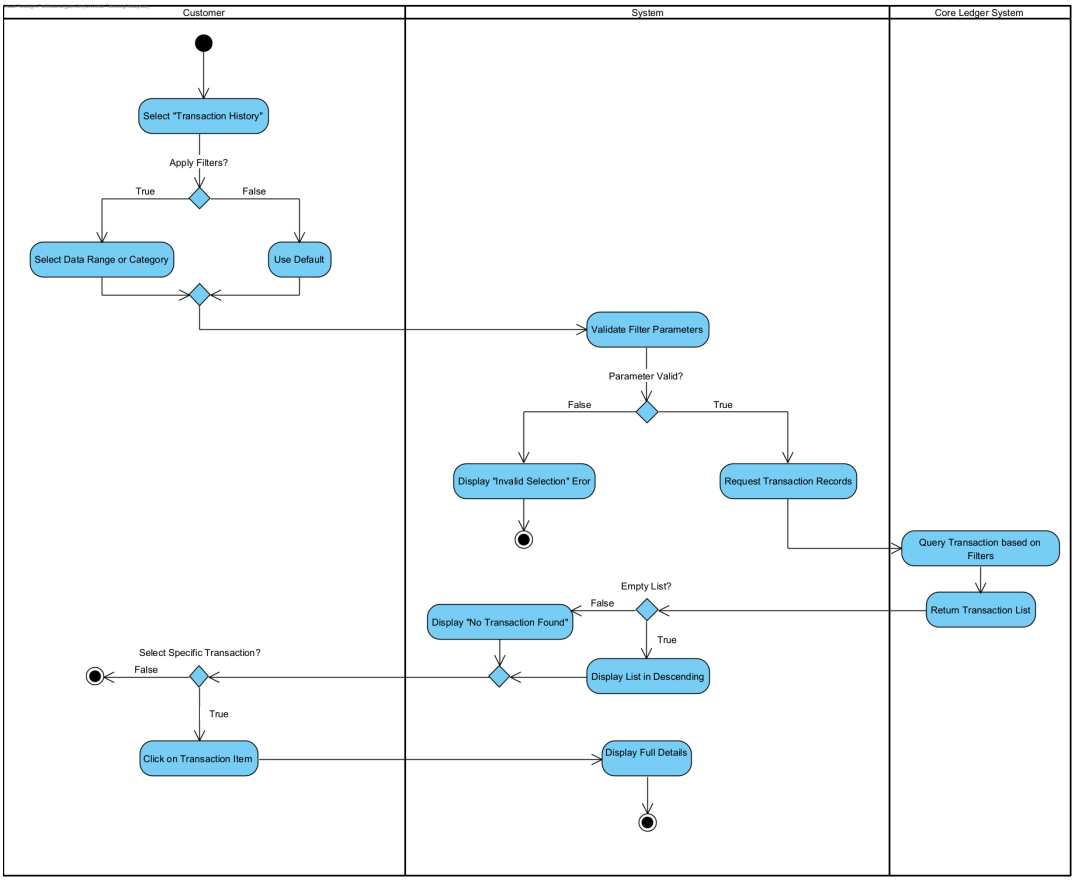


Figure 2.2.3: Activity Diagram for UC003 View Transaction History

For UC003, this diagram outlines the flow for reviewing past account activities. The Customer initiates this by selecting "Transaction History" and optionally applying filters such as date range or transaction category such as Payouts and Withdrawals. The system first validates these filter parameters.

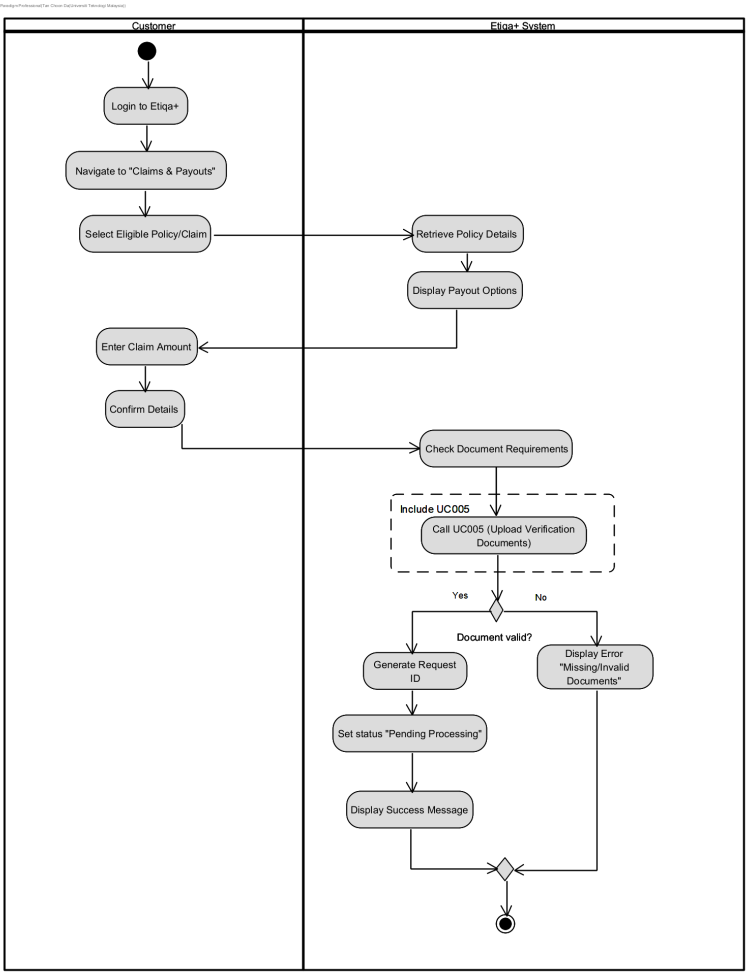


Figure 2.2.4: Activity Diagram for UC004 Request Payout

For UC004, this diagram illustrates how the user initiates the claiming payout process. Firstly, the flow begins with the customer logging into the Etiqa+ App, navigating to “Claims and Payout” and selecting an eligible policy. After that, the system will retrieve the policy details and display the payout options. Then, the customer needs to enter the claim amount and confirm the details. The system will check the document requirement by calling the UC005 (Upload Verification Documents). If the document is successfully uploaded and the request format is valid, the system will generate a request ID and set the status to “Pending”, concluding with a success message to the user. If the document is not valid, the system will display an error to the customer and end the request payout process.

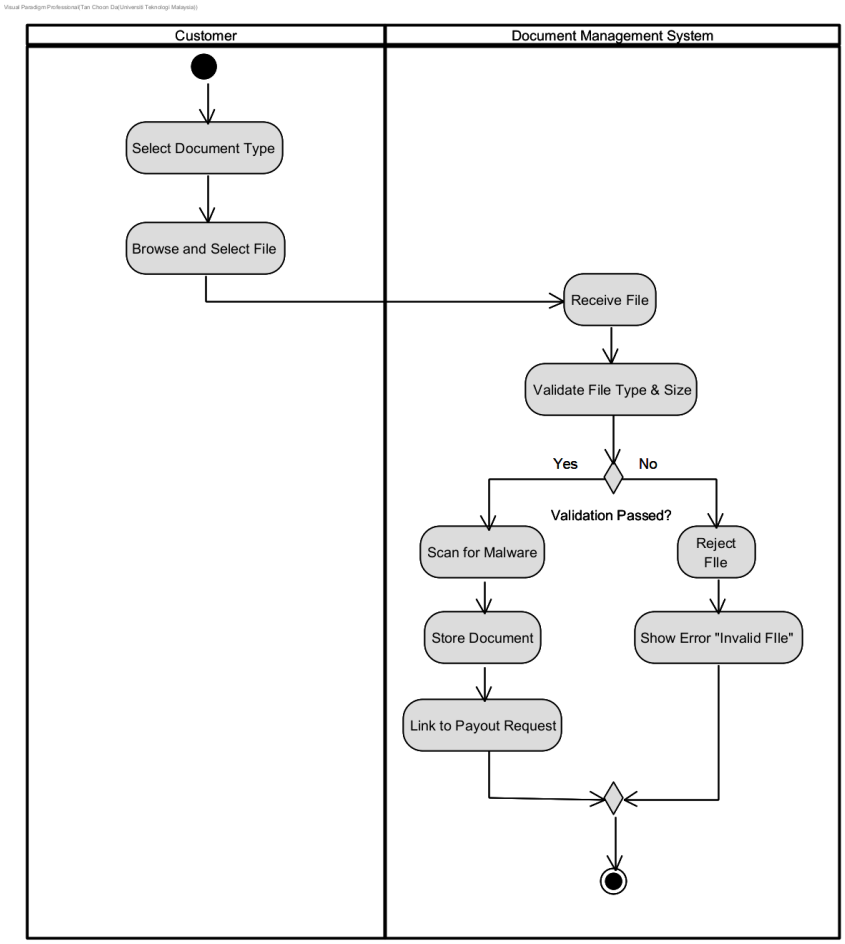


Figure 2.2.5: Activity Diagram for UC005 Upload Verification Document

For UC005, this diagram details the sub-process for handling file upload, which is invoked by the parent use case (UC004). The customer needs to select document type, browse and select the related file. After that, the system will receive the file and validate the file type and size. If any validation fails, the file is rejected with an error message. If all validation passes, the system will perform a malware scanning, document is securely stored and linked to the specific payout request ID , ensuring the claim has the necessary evidence for processing.

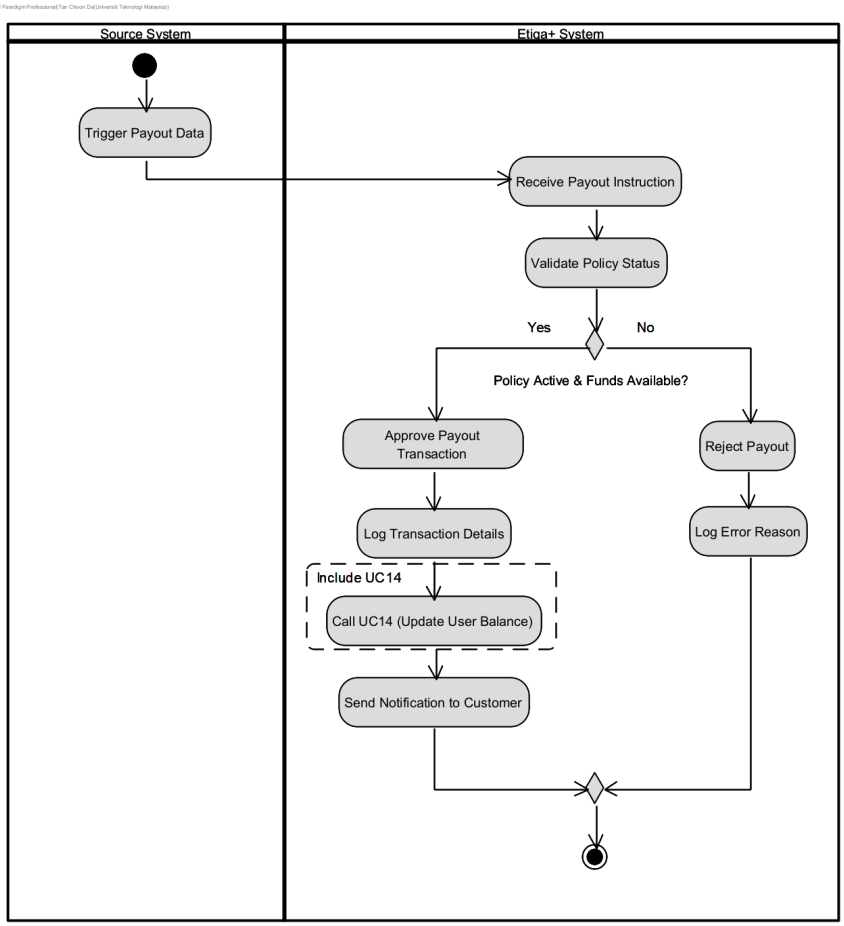


Figure 2.2.6: Activity Diagram for UC006 Process Incoming Payout

For UC006, this diagram represents the backend orchestration triggered by the Source System. Once a payout instruction is received, the Etiqa+ System validates the policy status and fund availability. If the policy is active and funds are available, the system will approve the transaction and explicitly calls UC14 (Update User Balance) to credit the customer's eWallet. Finally, a notification will be sent to the customer. If the policy is inactive or the funds are unavailable, the system will reject the payout and log the error reason for audit purposes.

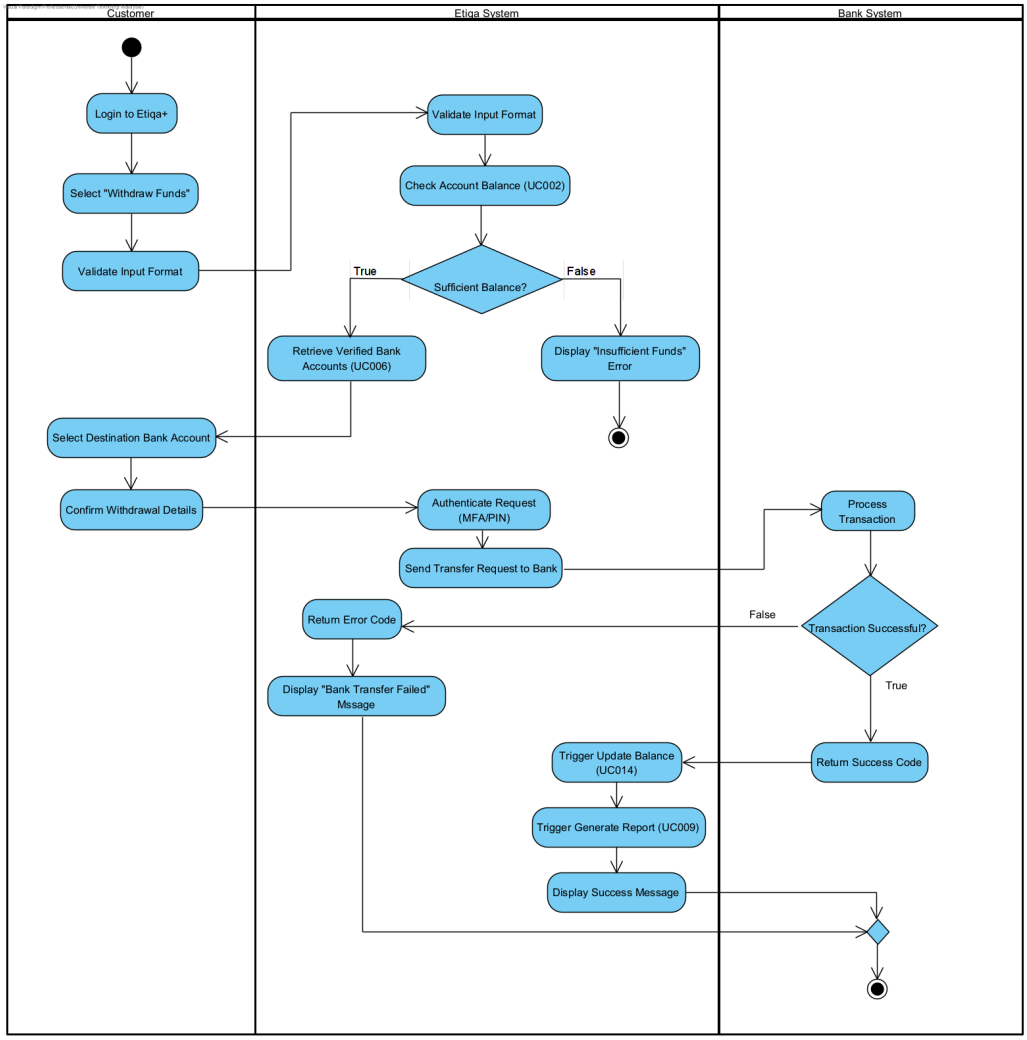


Figure 2.2.7: Activity Diagram for UC007 Withdraw Funds

For UC007, the withdrawal process led to the secure transfer of funds. It begins with the customer requesting an amount, followed by system validation of the account balance. If funds are sufficient, the user selects a pre-verified bank account. The system authenticates the user via MFA before transmitting the request to the external Bank System. Upon a successful bank response, the system triggers background processes to update the user's balance and generate a transaction report, finally displaying a success message to the customer.

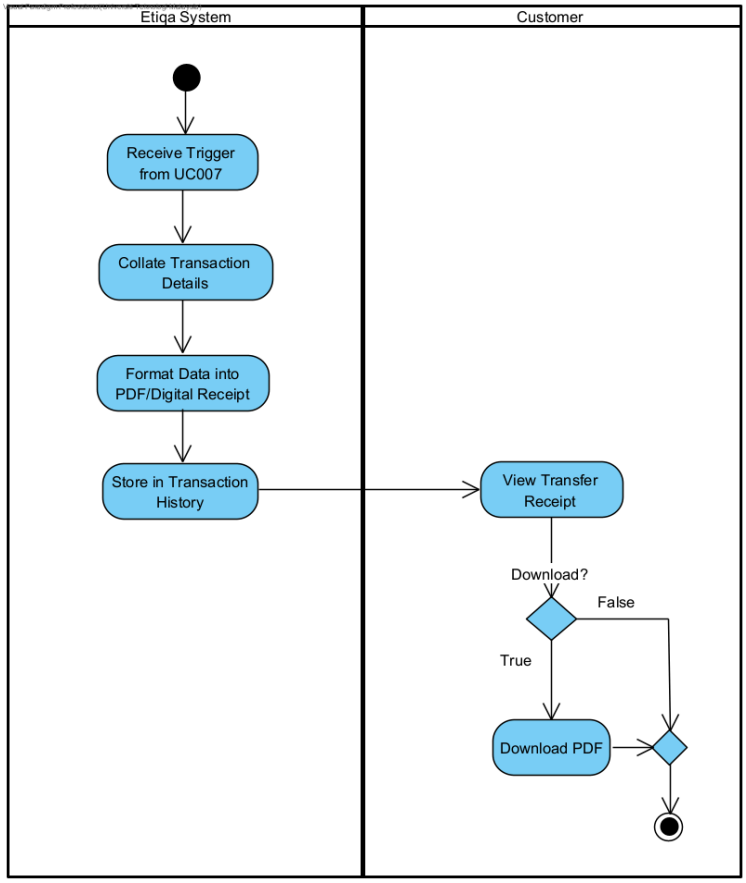


Figure 2.2.8: Activity Diagram for UC009 Generate Transfer Report

For UC009, it serves as a digital receipt mechanism, this process is triggered post-transaction. The system aggregates all critical transaction metadata, including the timestamp, amount, and destination details. This data is formatted into a downloadable PDF or digital view. The generated report is then stored in the customer's transaction history. This allows the user to access, view, and download proof of the transfer at any time, fulfilling the requirement for transparency.

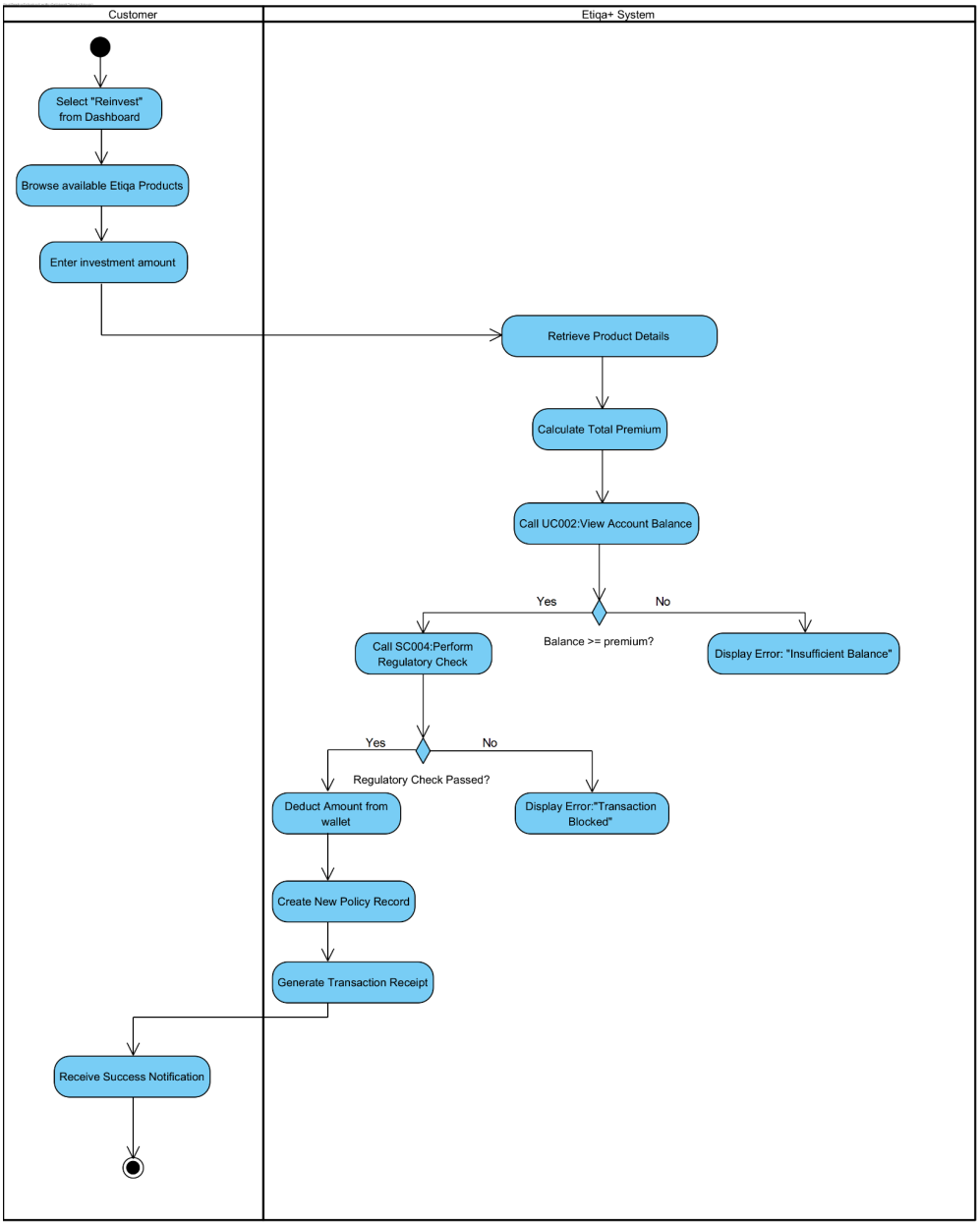


Figure 2.2.9: Activity Diagram for UC010 Reinvest Product

For UC010, this use case is designed to create a seamless experience where customers can easily put their payout funds back to work. Instead of waiting for external bank transfers, users can instantly use their eMoney balance to purchase new insurance or Takaful plans directly. The system handles the heavy lifting in the background.It first checks if the customer is eligible for the plan and confirms they have enough funds. Before the transaction is finalized, the system automatically performs a regulatory check to ensure the purchase complies with strict financial guidelines. The result is an instant, secure update to their coverage and wallet balance, making it easier for customers to stay protected.

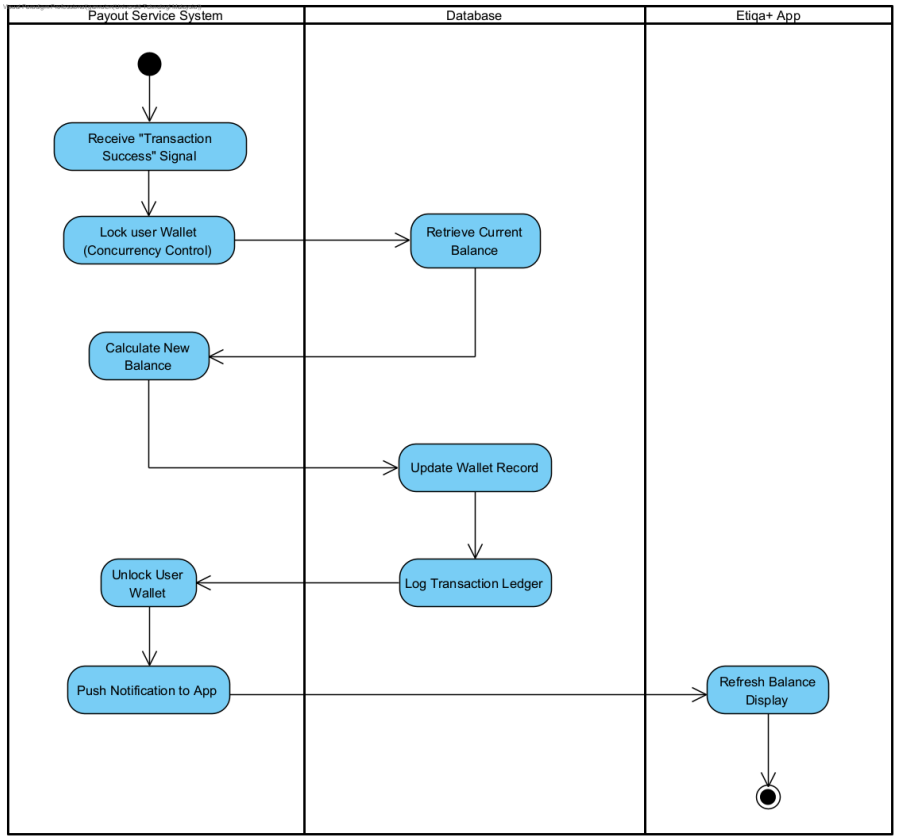


Figure 2.2.10: Activity Diagram for UC014 Update User Balance

For UC014, this automated backend process ensures financial consistency following a transaction. Triggered immediately by a successful withdrawal, the system locks the user's wallet to prevent concurrent modifications. It calculates the new balance by subtracting the withdrawal amount from the current total and updates the database record. Once the ledger is updated, the wallet is unlocked, and a push notification is sent to the customer's application to refresh the displayed balance, ensuring the user sees accurate data in real-time.

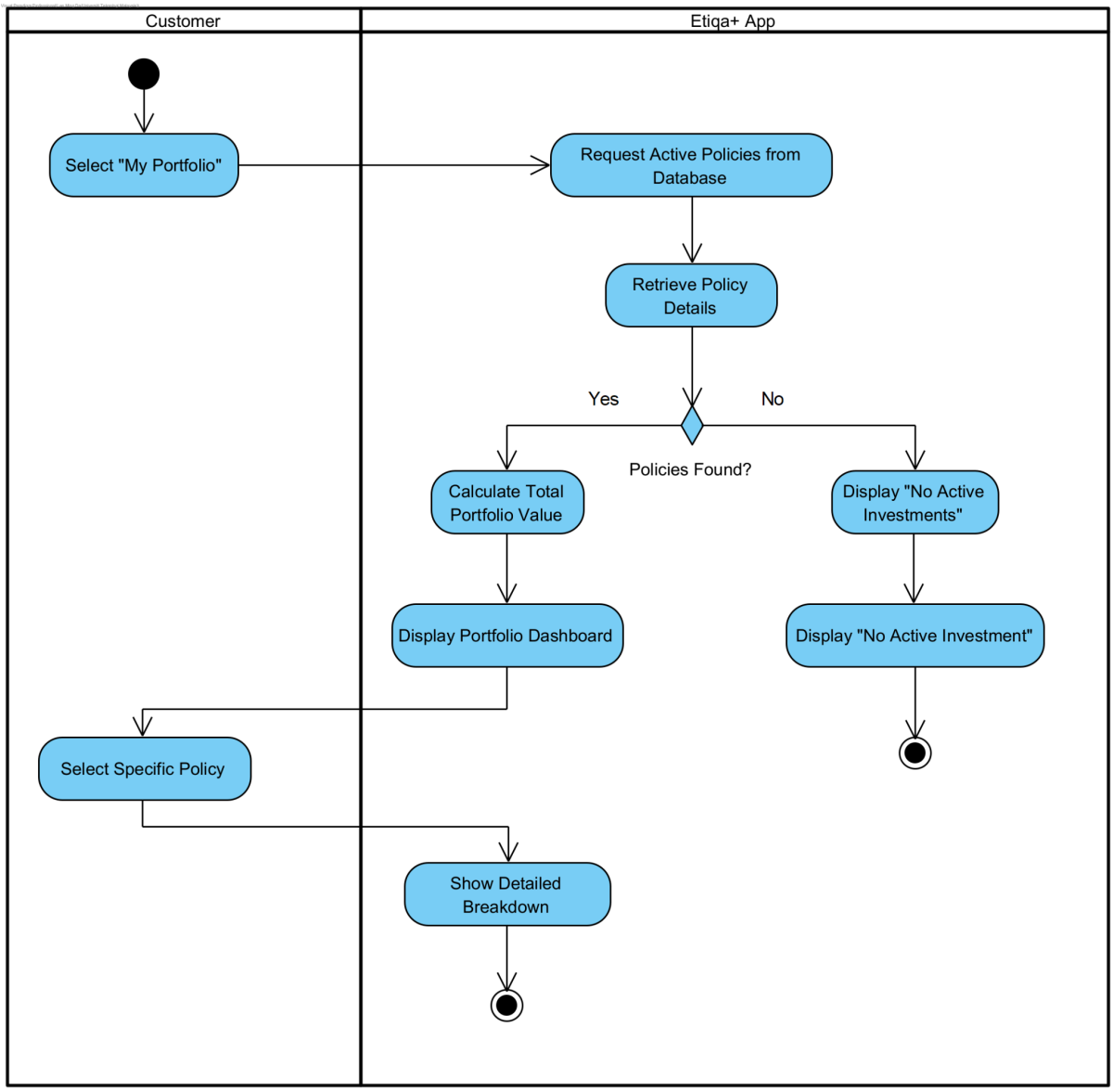


Figure 2.2.11: Activity Diagram for UC015 View Policy Portfolio

For UC015, this use case is designed to give customers a comprehensive, real-time overview of their insurance and Takaful assets directly through the Etiqa+ app. Upon accessing the portfolio section, the system automatically queries the central database to retrieve all active policies linked to the user's profile. It then aggregates these details to calculate the total current value of their holdings, presenting the data in a unified dashboard that allows users to drill down into specific policy coverage and premiums. This consolidated view ensures customers can effortlessly monitor their investment performance and protection status without needing to track multiple physical documents.

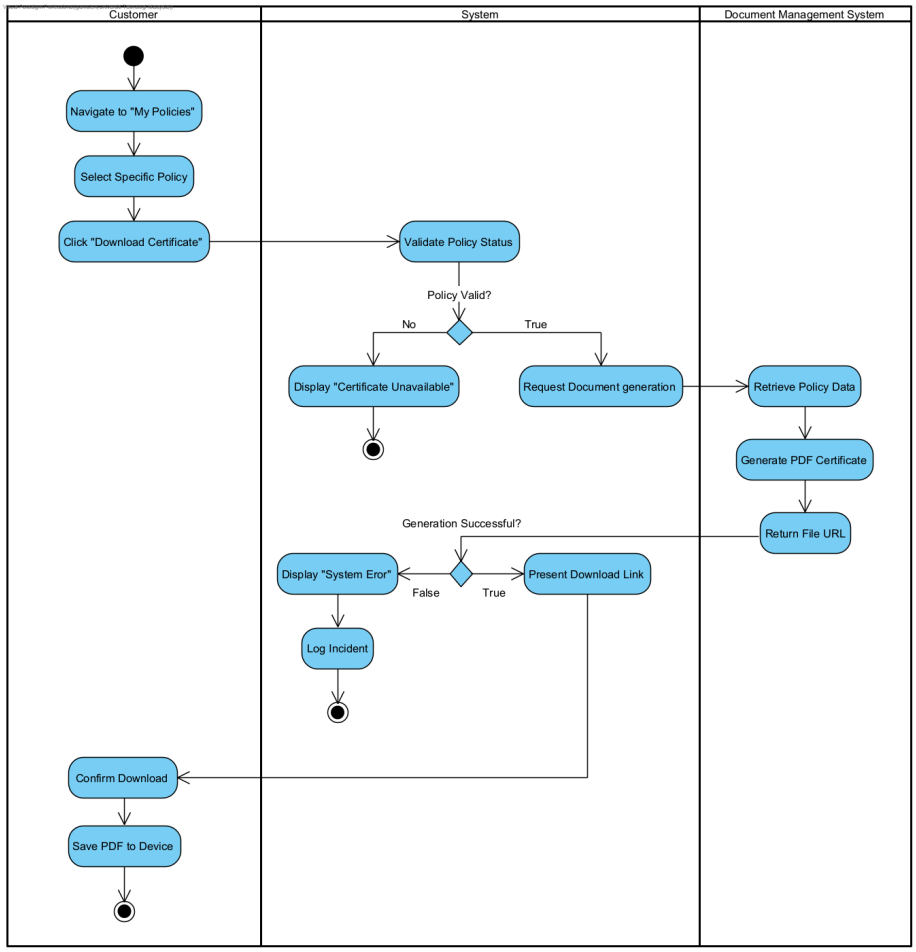


Figure 2.2.12: Activity Diagram for UC016 Download Policy Certificate

For UC016, it details the sub-process for retrieving official policy documentation, which is initiated by the Customer from their dashboard. The Customer needs to navigate to the "My Policies" section, select the relevant active policy, and click the download button. After that, the System will validate the policy status to ensure it is eligible for certificate generation. If the policy is valid, the System triggers the backend to retrieve the data and generate a PDF file. If the generation is successful, the System presents the file to the Customer to ensure they possess a digital copy of their coverage evidence for personal record-keeping or claims purposes.

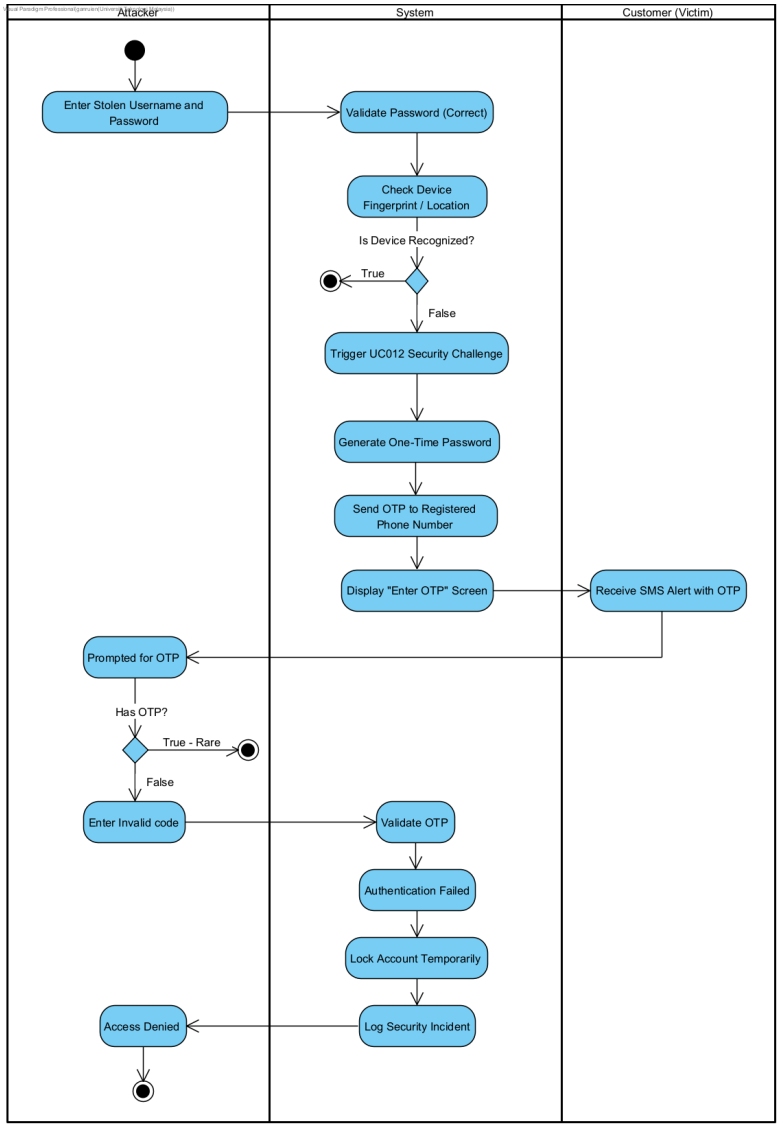


Figure 2.2.13: Activity Diagram for SC001 Block Attack

For SC001, this diagram details the defensive sub-process for handling suspicious login attempts, which is invoked by the security logic in UC012 Update Security Settings. The Attacker needs to attempt a login using stolen credentials from an unrecognized device. After that, the System will validate the password but simultaneously check the device fingerprint. If the device is not recognized, the system will trigger a security challenge by generating a One-Time Password and sending it to the legitimate Customer's mobile number. The Attacker is prompted to enter this OTP. If the Attacker fails to provide the valid code, the system denies access, locks the account, and logs the security incident, ensuring the threat is neutralized.

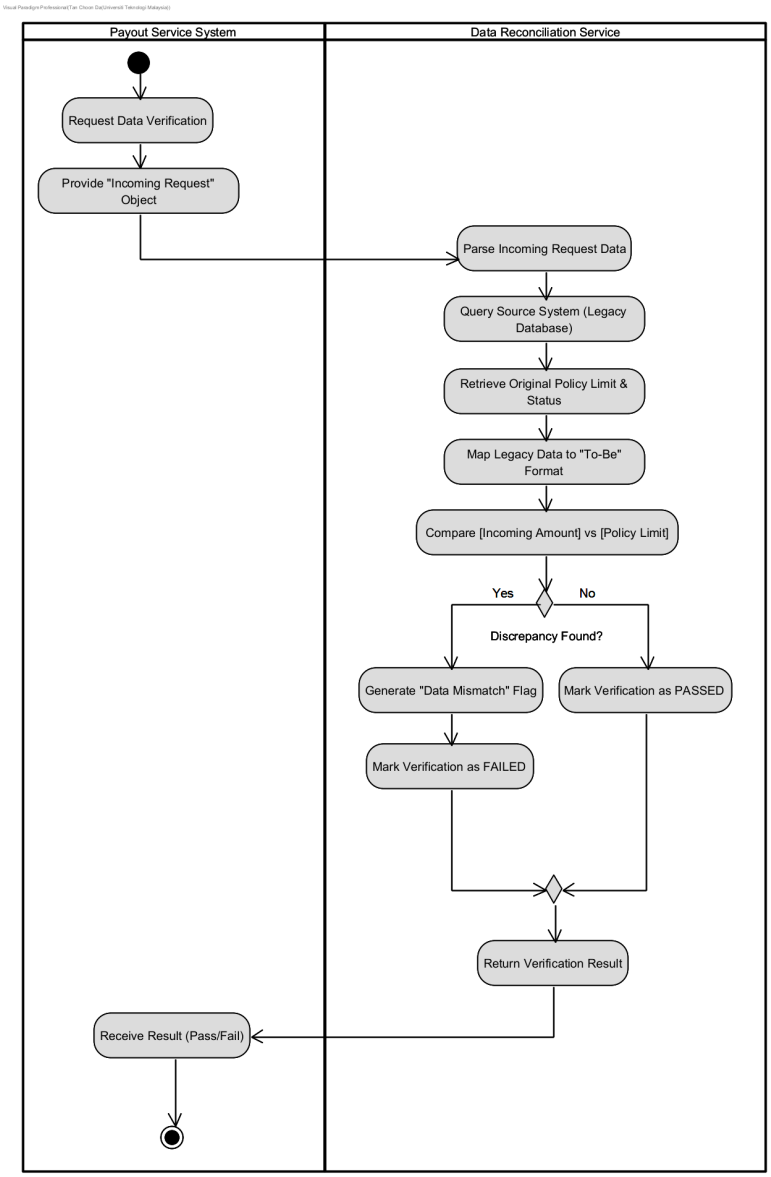


Figure 2.2.14: Activity Diagram for SC002 Consolidate Payout Data

For UC013, this diagram explains the way used to mitigate fraud, it shows how the system decides if a claim is fraudulent. The Data Reconciliation Service fetches the "Sources of Truth" from the legacy database and compares it against the "Incoming Request" data. It explicitly checks for discrepancies such as claim amount exceeding the policy limit or mismatched user ID. If the discrepancies are found, the data reconciliation service will generate a "Data Mismatch" flag and mark the verification as "Failed". If there are no discrepancies found, the system will mark the verification as "Passed". After the decision point, the data reconciliation system will return the verification result to the payout service system.

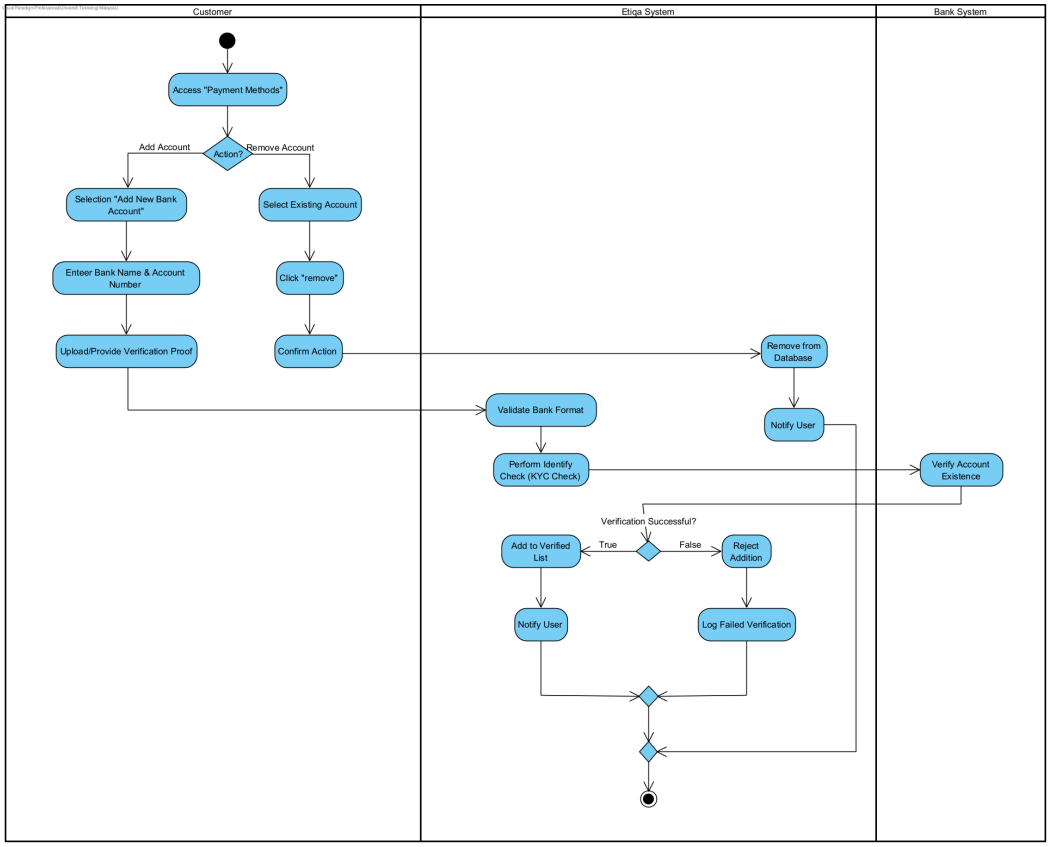


Figure 2.2.15: Activity Diagram for SC003 Manage Verified Bank Accounts

For SC003, this mitigation use case manages the security of external destinations. When a customer attempts to add a new bank account, the system performs a rigid validation process, including format checks and identity verification (KYC) with the external bank. Only if these checks pass then the account transitions to a "Verified" state. Conversely, users can remove existing accounts. This strict management ensures that funds can only be withdrawn to legitimate, confirmed destinations, preventing redirection to unauthorized accounts.

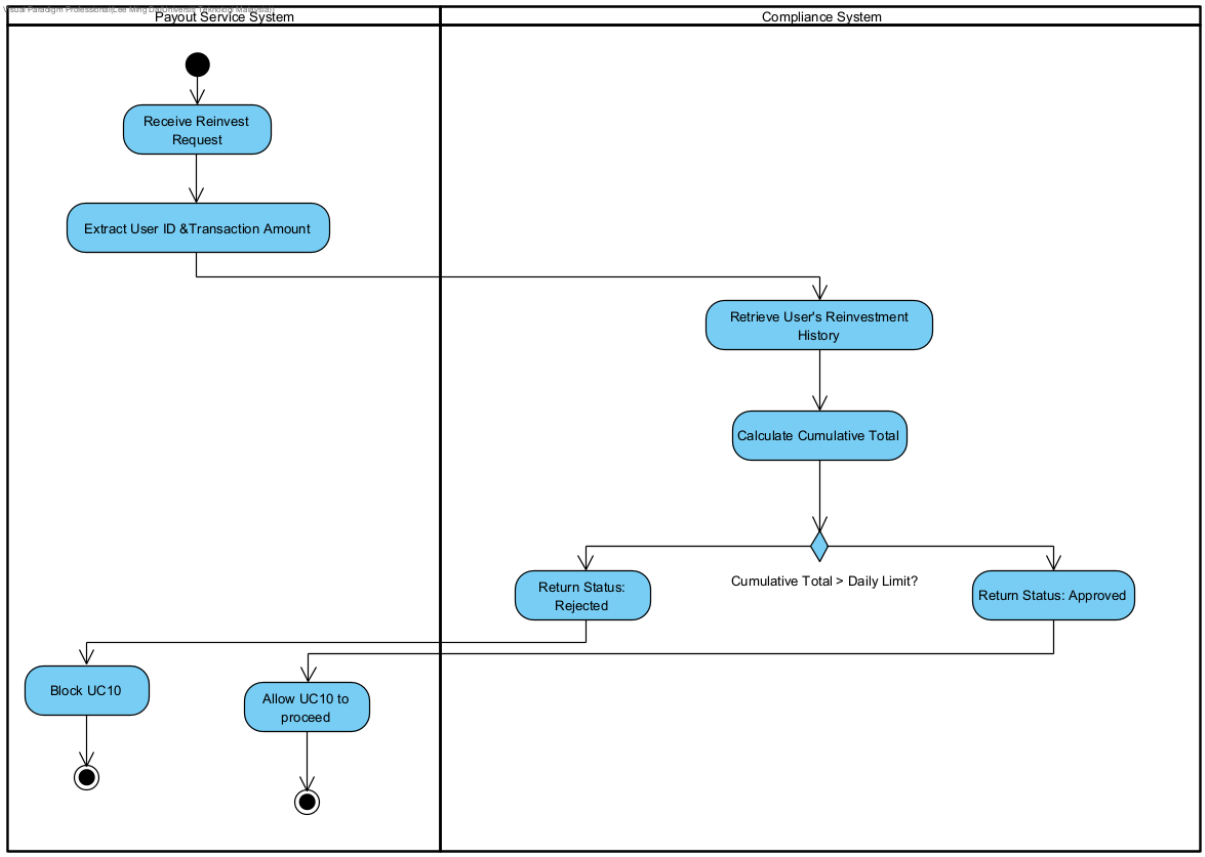


Figure 2.2.16: Activity Diagram for SC004 Perform Regulatory Checking

SC004: Perform Regulatory Checking This use case functions as the platform's automated compliance guardian, executing invisibly in the background during critical financial transactions such as reinvestments. Its primary role is to enforce Bank Negara Malaysia's strict regulatory standards by instantly cross-referencing transaction data against global Anti-Money Laundering (AML) and Counter-Financing of Terrorism (CFT) watchlists. Additionally, the system evaluates the activity against the Risk Management in Technology (RMIT) framework to detect any potential fraud or structuring patterns. If a risk threshold is breached or a watchlist match is found, the system immediately halts the transaction, ensuring the platform remains secure without disrupting legitimate user activities.

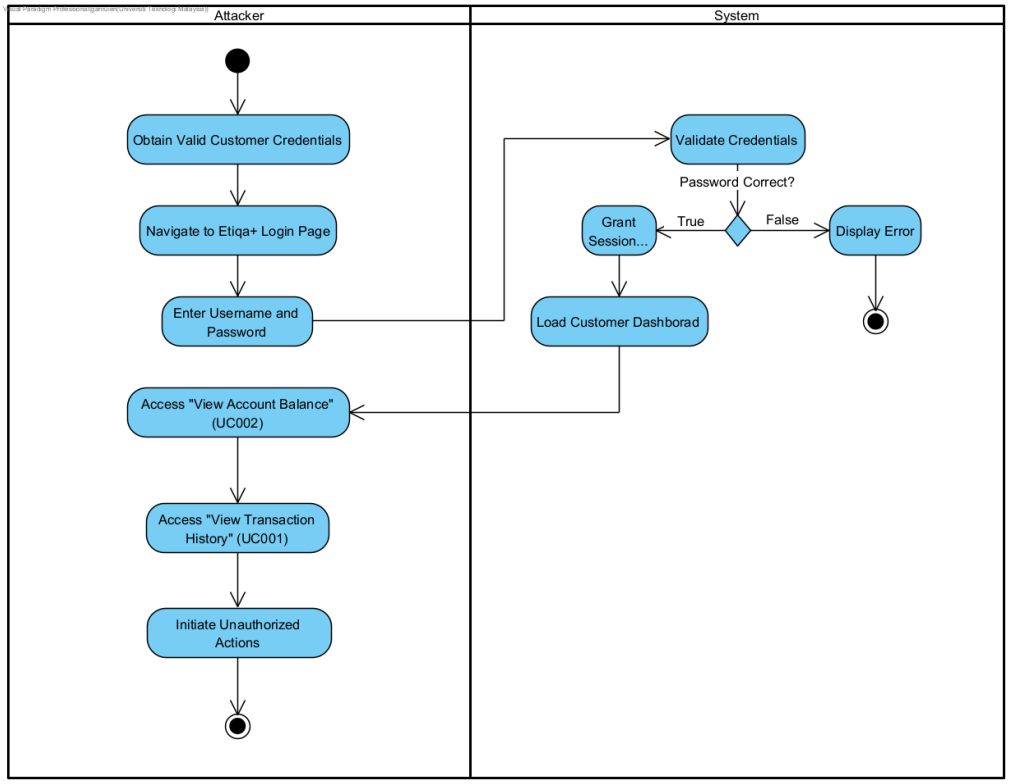


Figure 2.2.17: Activity Diagram for MUC01 Account Takeover

For MUC01, this diagram shows how an Attacker and the System communicate. In order for the system to validate the password, the attacker must first get legitimate credentials and enter them on the login screen. The system loads the dashboard and provides a session if the credentials are correct. This effectively gives the attacker access to private data, including account balances and transaction histories, and enables them to carry out unlawful acts.

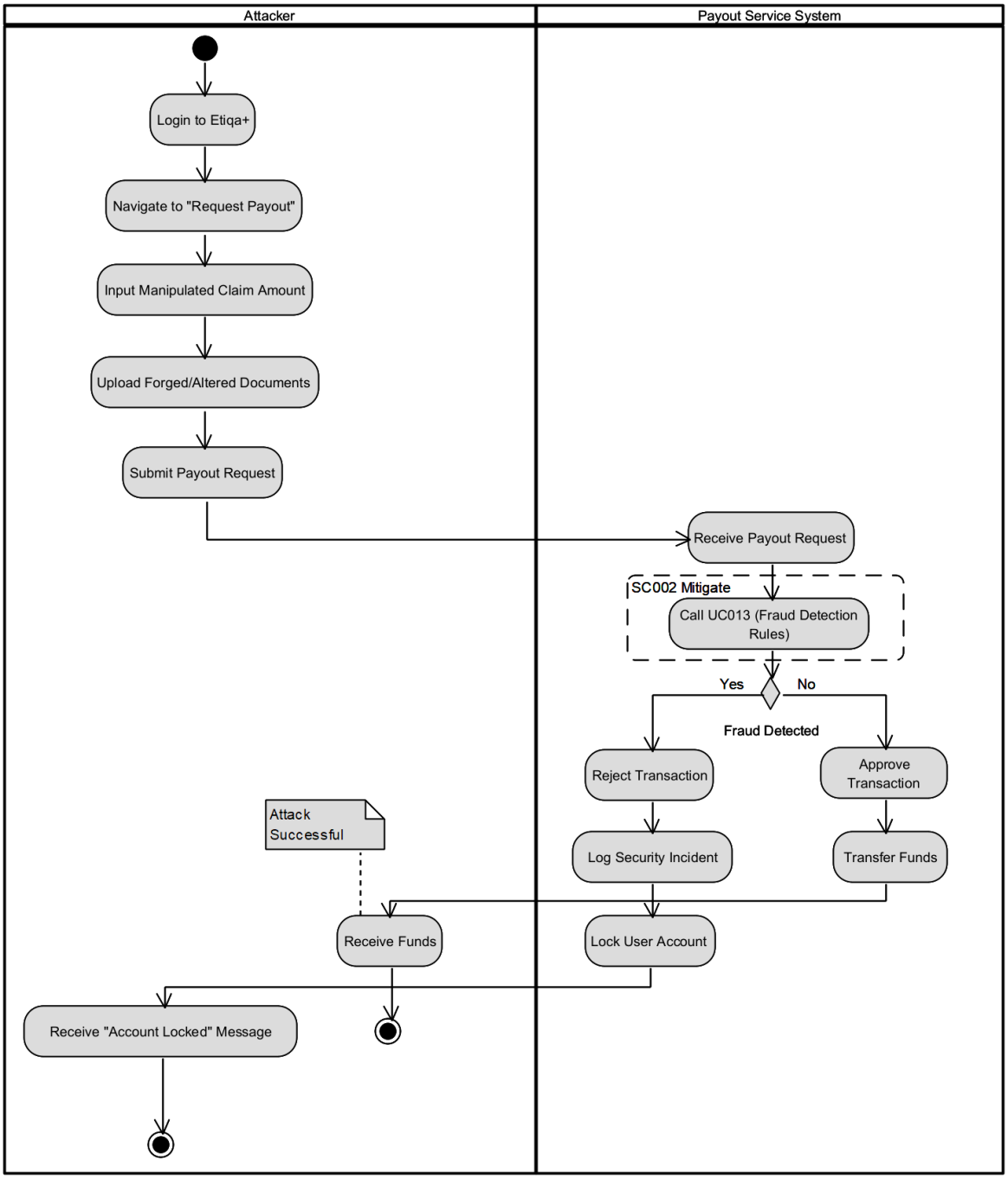


Figure 2.2.18: Activity Diagram for MUC02 Submit Fraudulent Claim

For MUC02, this diagram visualizes the "threat" scenario from the perspective of an attacker. The attacker attempts to exploit the system by inputting manipulated claim amounts and uploading forged documents via the "Request Payout" interface. The system counters this by triggering the fraud detection rules by utilizing UC013. If the fraud is detected, the transaction is rejected, the security incident is logged and the user account will be locked. If the system fails to detect the discrepancy, then it will approve the transaction for the attacker and the attacker will receive the transaction.

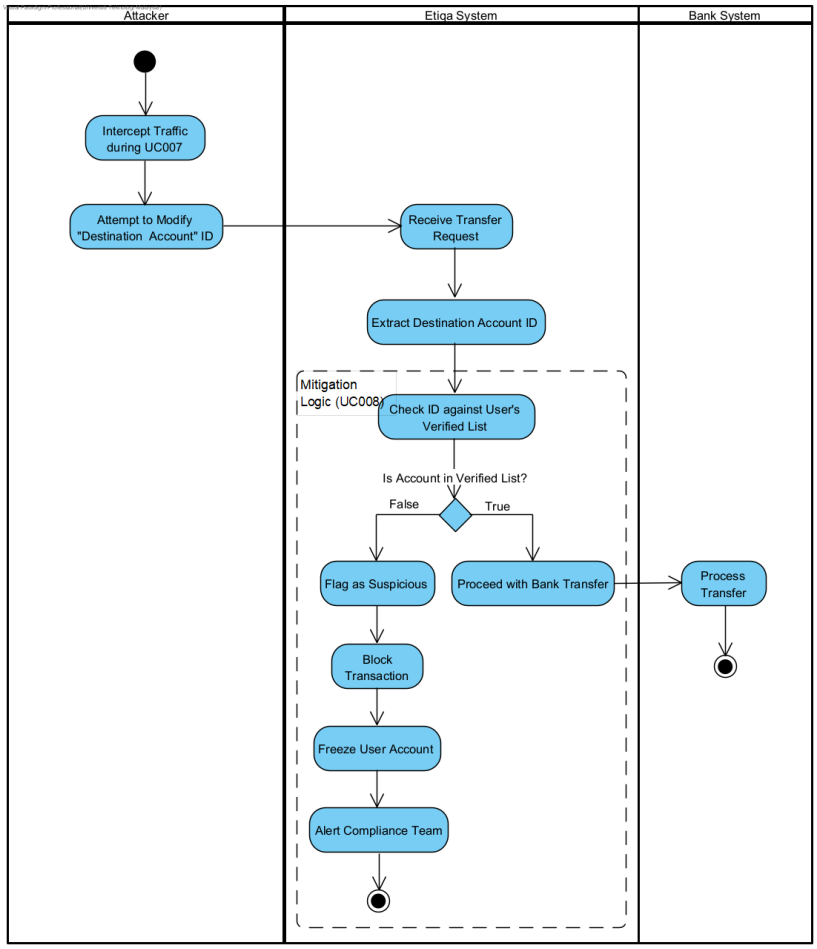


Figure 2.2.19: Activity Diagram for MUC03 Intercept Fund Transfer

For MUC003, this diagram illustrates a security threat where an attacker attempts to intercept a withdrawal request and alter the destination account ID. The mitigation logic embedded in the system acts as a firewall. Before processing any transfer, the system cross-references the requested destination ID against the user's pre-approved "Verified List." If the account is not found in this safe list, the transaction is immediately blocked, the user's account is frozen, and a security alert is logged in order to effectively neutralize the theft attempt.

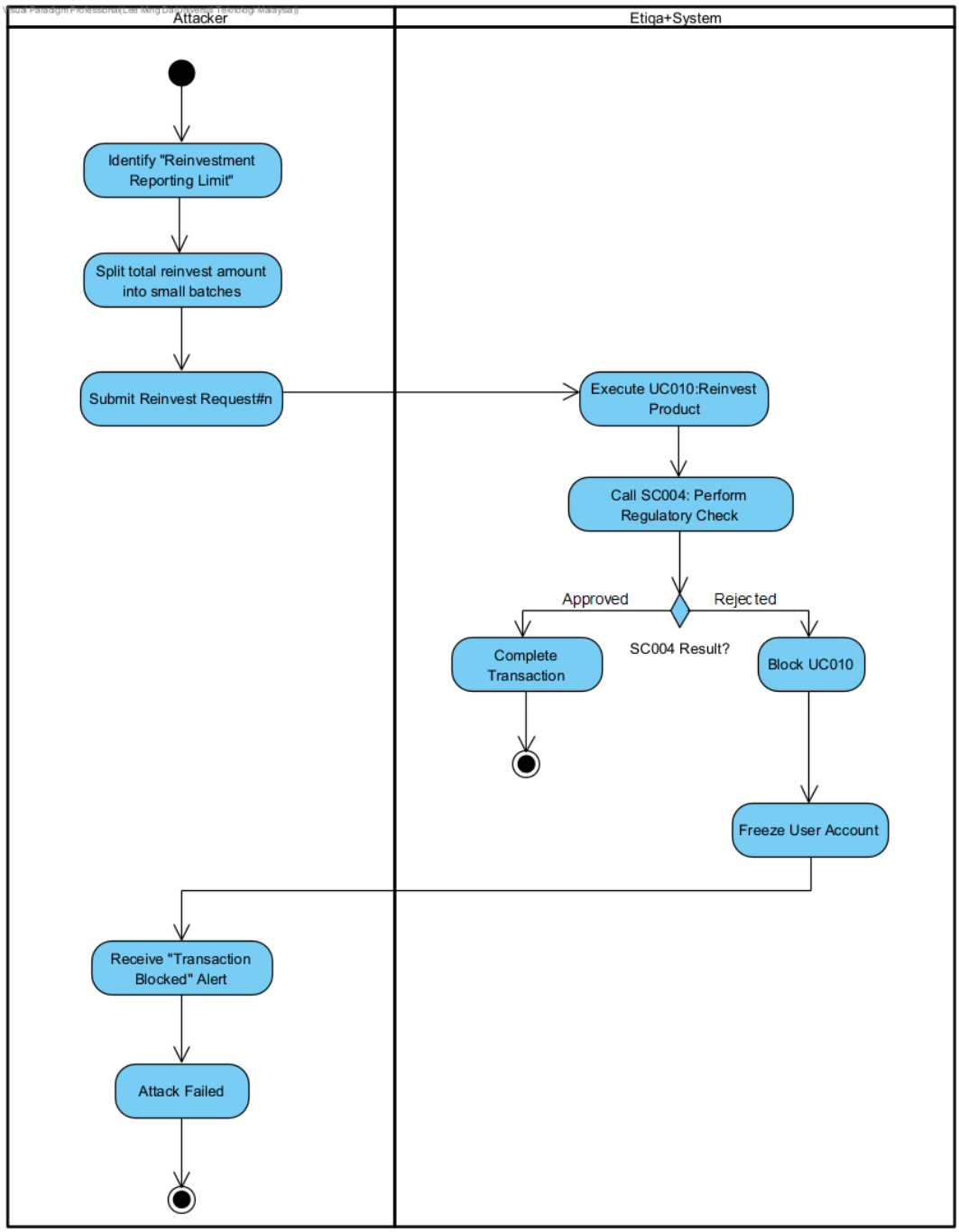


Figure 2.2.20: Activity Diagram for MUC04 Bypass Compliance Check

For MUC04, this misuse case describes a structuring attempt where a malicious attacker tries to circumvent regulatory reporting limits. The attacker intentionally splits a large transaction value into multiple smaller, inconspicuous amounts to avoid triggering the automated high-value alerts. This scenario highlights the necessity of the system's advanced monitoring logic, which must detect cumulative transaction patterns over time rather than just isolated events, ensuring that illicit funds cannot flow through the Etiqa ecosystem undetected

3. DATA PERSPECTIVE: UML DOMAIN CLASS DIAGRAM (DCD)

The UML Domain Class Diagram (DCD) illustrates the static view of the Etiqa eMoney System by modeling the underlying data structures, attributes and the relationships between them. This diagram identifies the core business entities required to support the functional use cases such as customer, eWalletAccount, Payout Request, Transaction, ComplianceLog and SecurityLog. This diagram captures the essential data requirements for the system, mapping directly to the business rules defined in the URD.

The Domain Class Diagram illustrates the static data structure of the Etiqa eMoney System, centering on the Customer entity which maintains a strictly one-to-one relationship with an EWalletAccount to ensure a unified ledger for managing the currentBalance. Customers can initiate PayoutRequests, which are fundamentally linked to a single Policy as the source of truth and are composed of multiple Documents that serve as mandatory evidence. The successful requests will generate Transaction that are verified against ComplianceLogs to ensure regulatory adherence while the system concurrently tracks the user behavior through SecurityLogs to monitor for potential threats or anomalies.

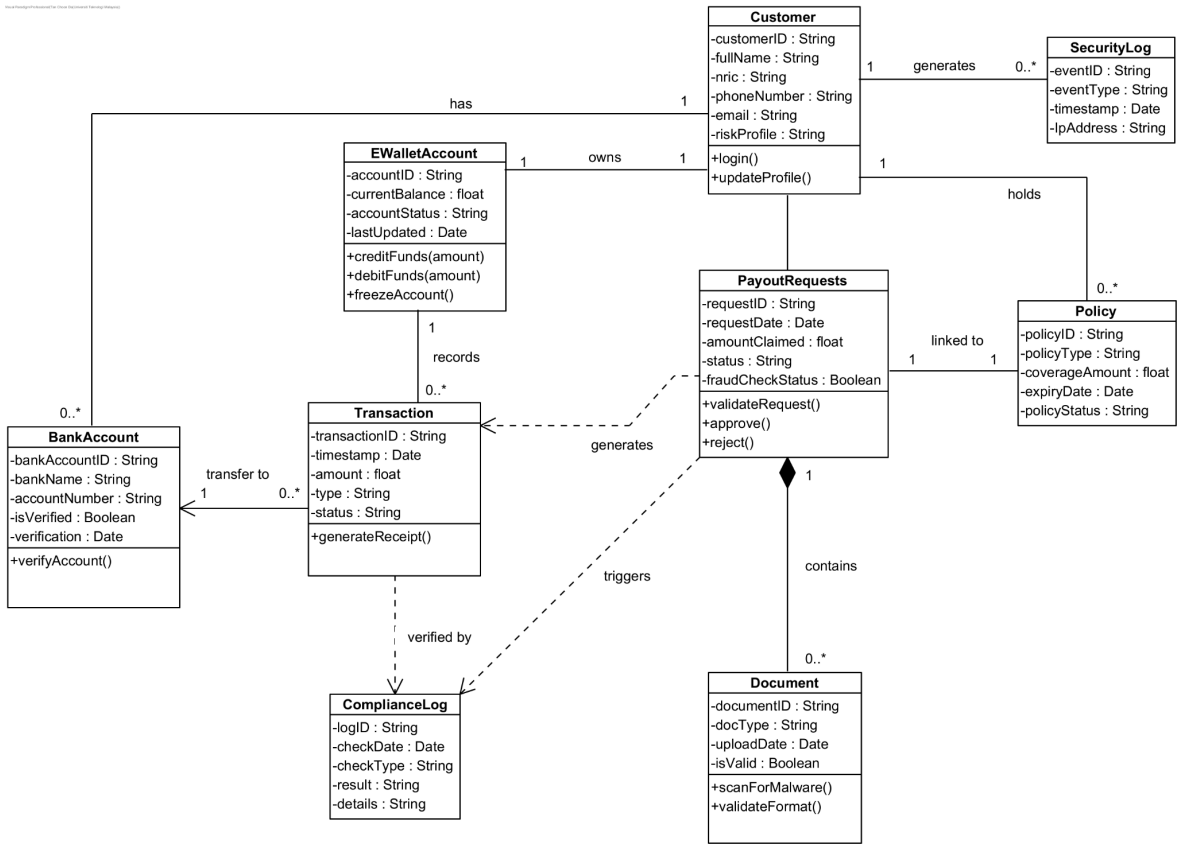


Figure 3.0: Domain Class Diagram for Etiqa Digital Payout

3.1 Classes

- Customer
    - Attributes: customerID, fullName, nric, phoneNumber, email, riskProfile
    - Operations: login(), updateProfile()
- EWalletAccount
    - Attributes**:** accountID, currentBalance, accountStatus, lastUpdated
    - Operations: creditFunds(amount), debitFunds(amount), freezeAccount()
- PayoutRequests
    - Attributes: requestID, requestDate, amountClaimed, status, fraudCheckStatus
    - validateRequest(), approve(), reject()
- Policy
    - Attributes: policyID, policyType, coverageAmount, expiryDate, policyStatus
- Document
    - Attributes: documentID, docType, filePath, uploadDate, isValid
    - Operations: scanForMalware(), validateFormat()
- Transaction
    - Attributes: transactionID, timestamp, amount, type, status
    - Operations: generateReceipt()
- BankAccount
    - Attributes: bankAccountID, bankName, accountNumber, isVerified, verificationDate
    - Operations: verifyAccount()
- ComplianceLog
    - Attributes: logID, checkDate, checkType, result, details
- SecurityLog
    - Attributes: eventID, eventType, timestamp, ipAddress

3.2 Relationships

3.2.1 Associations

Customer – EWalletAccount

- A Customer owns 1 EWalletAccount.
- Each EWalletAccount is owned by 1 Customer.

Customer – BankAccount

- A Customer has 0..* BankAccounts.
- Each BankAccount is linked to 1 Customer (implied via ownership context, though the diagram shows direct line from Customer to BankAccount).

Customer – Policy

- A Customer holds 0..* Policies.
- Each Policy is held by 1 Customer.

Customer – PayoutRequests

- A Customer initiates 1 PayoutRequest (per specific interaction context, though typically a customer can make multiple requests over time, the diagram shows a `1` to `0..*` relationship logic via initiation flow). *Note: The diagram shows Customer 'initiates' PayoutRequest.*

Customer – SecurityLog

- A Customer generates 0..* SecurityLogs.
- Each SecurityLog is generated by 1 Customer.

PayoutRequests – Policy

- A PayoutRequest is linked to 1 Policy.
- Each Policy can be linked to 0..* PayoutRequests (diagram shows 1 to 1 link, implying one request per specific policy claim context).

PayoutRequests – Transaction

- A PayoutRequest generates 1 Transaction.
- Each Transaction is generated by 1 PayoutRequest (implied).

PayoutRequests – ComplianceLog

- A PayoutRequest triggers 1 ComplianceLog verification (implied dependency/trigger relationship).

EWalletAccount – Transaction

- An EWalletAccount records 0..* Transactions.
- Each Transaction is recorded by 1 EWalletAccount.

Transaction – BankAccount

- A Transaction transfers to 0..* BankAccounts (typically 1 for a withdrawal).
- Each BankAccount is associated with 1 Transaction context (diagram shows 1 to 0..* from BankAccount to Transaction).

Transaction – ComplianceLog

- A Transaction is verified by 1 ComplianceLog.

3.2.2 Composition

PayoutRequests – Document

- A PayoutRequest contains 0..* Documents.
- Each Document belongs to 1 PayoutRequest (The black diamond indicates Composition, meaning Documents cannot exist without the Request).

4. BEHAVIORAL PERSPECTIVE: UML STATE-TRANSITION DIAGRAM (STD)

A UML State-Transition Diagram (STD) is an object's lifecycle using specific symbols to represent behavior changes. The flow begins at the Initial State which is represented by a solid filled circle and concludes at the Final State which is represented by a solid circle within a hollow one. States that are represented by rounded rectangles define distinct phases where an object waits or performs activities. Transitions connect these states using solid arrows to dictate how the object moves from one phase to another. These arrows are labeled with the syntax "Event [Condition] / Action". In this format, the "Event" is the trigger, the "Condition" is a boolean guard that must be true for the transition to proceed, and the "Action" describes the operation executed during the shift. This notation strictly governs the logic of the system's behavioral changes.

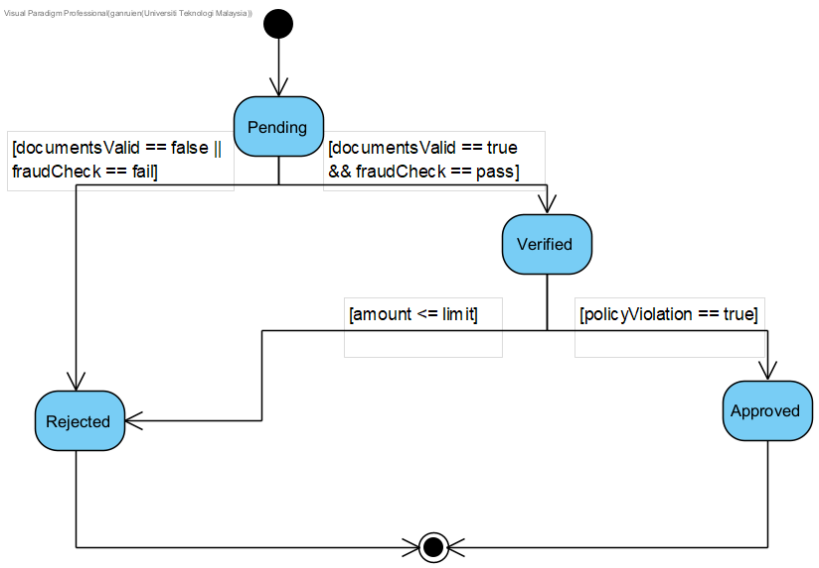


Figure 4.1: State Transition Diagram for Payout Request

The PayoutRequest entity manages the lifecycle of a customer's claim. It begins in a Pending state upon creation. The system immediately triggers validation processes. If the documents and fraud checks pass, it transitions to Verified. If any check fails, it moves directly to Rejected. From the Verified state, the request awaits a final decision of successful authorization transitions it to Approved, triggering the financial transaction, while a policy violation or manual denial moves it to Rejected.

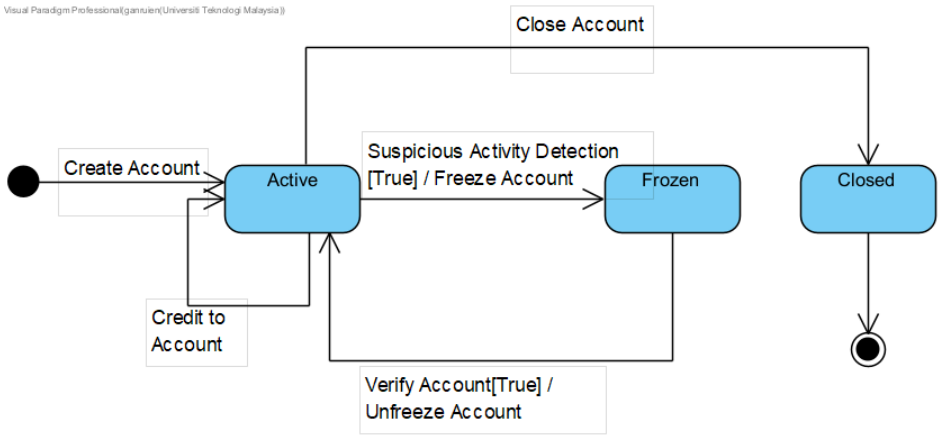


Figure 4.2: State Transition Diagram for e-Wallet Account

The e-Wallet Account entity represents the user's operational fund status. It starts in an Active state that allows standard deposits and withdrawals. If the system detects suspicious behavior, the account transitions to Frozen, temporarily disabling outgoing funds until a verification process restores it to Active. Finally, if a user decides to leave and the balance is zero, the account transitions to Closed, archiving the data and preventing further use.

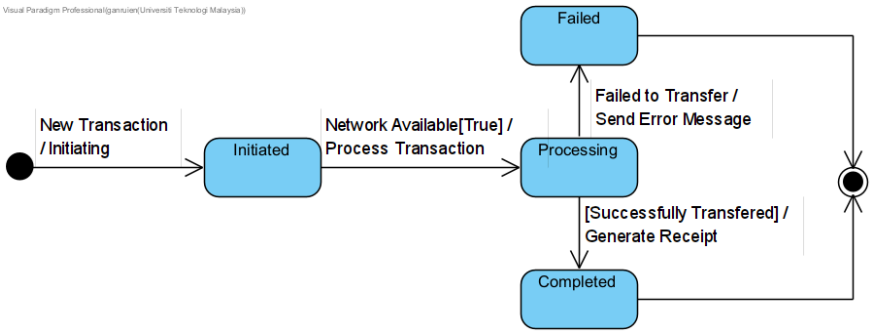


Figure 4.3: State Transition Diagram for Transaction

The Transaction entity tracks the specific movement of funds. It initializes in an Initiated state and quickly moves to Processing as the system communicates with the external banking network. This state represents a critical waiting period. Depending on the response from the bank, the transaction definitively transitions to either Completed and generate a receipt or Failed when error occurs to ensure that the financial status is always atomic and clear.

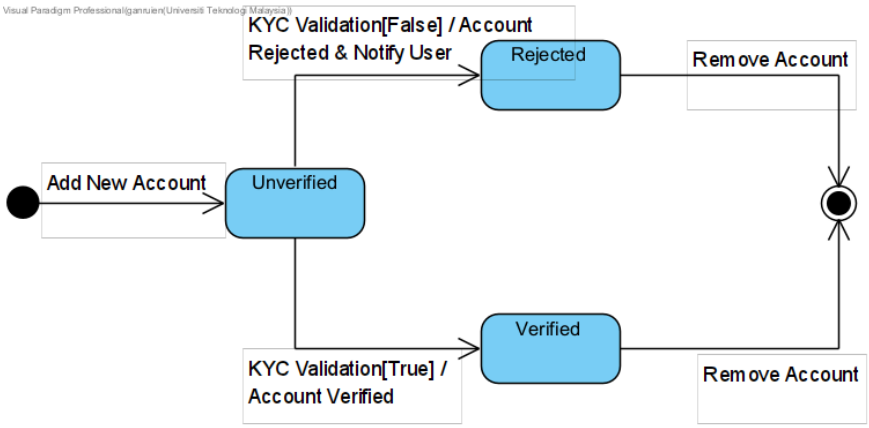


Figure 4.4: State Transition Diagram for Bank Account

The Bank Account entity handles the verification of external withdrawal destinations. A new account starts as “Unverified”. It must typically undergo an identity check which is KYC and bank validation. A match transitions the state to Verified to enable withdrawals meanwhile a mismatch results in a Rejected state. Users can also delete an account at any time, which terminates the entity's lifecycle regardless of its current status.

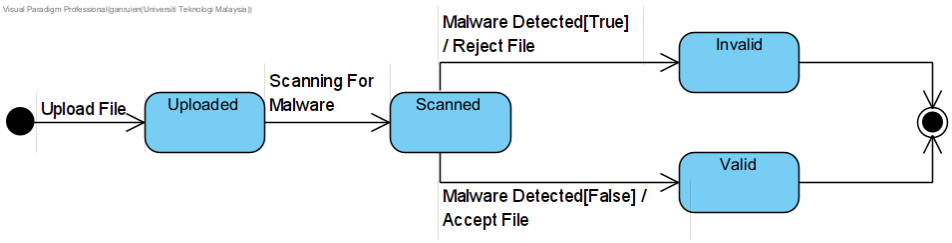


Figure 4.5: State Transition Diagram for Document

The Document entity tracks the file validation process for claims. Upon upload, a file enters the Uploaded state. It automatically transitions to Scanned as the system performs anti-malware checks. Based on the scan results and format compatibility, the document ultimately settles as either Valid or Invalid to ensure only safe data enters the system's processing workflow.

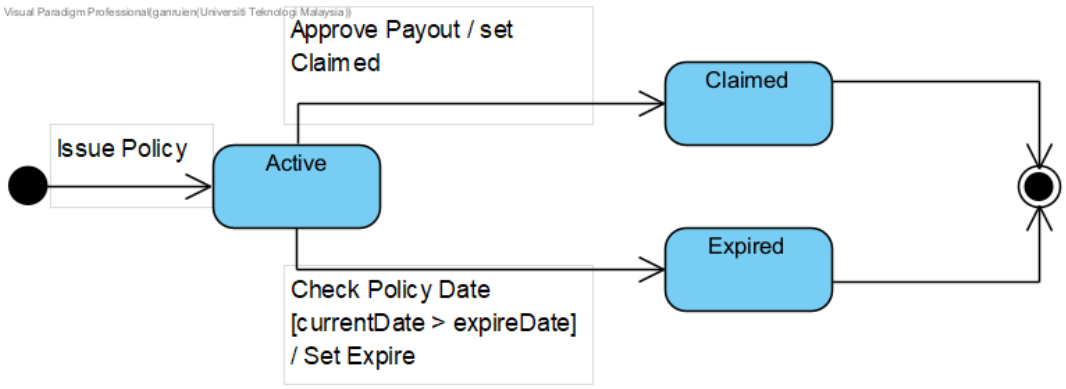


Figure 4.6: State Transition Diagram for Policy

The Policy entity represents the insurance product's status. It begins as Active when issued to the customer. The system monitors the policy against two conditions which are time and utilization. If the current date passes the policy's expiry date without a claim, it transitions to Expired. Alternatively, if a valid claim payout equals the total coverage amount, it transitions to Claimed.

5. CONCLUSION

In this project 2, we have transitioned from the textual requirement defined in the project 1 (User Requirements Document) to a comprehensive set of visual technical models. The integration of User Case Diagram, Activity Diagram, Domain Class Diagram and State Transition Diagram provides a holistic view of the system, ensuring that both the user experience and backend logic are defined clearly for the development phase.

Challenges faced during modelling process:

1. Modeling security and misuse cases: One of the complex tasks was integrating security requirements into the functional models without cluttering them. Determining the correct relationships for the Attacker in the Use Case Diagram required careful analysis and distinguishing between standard system errors and intentional malicious.
2. Defining system boundaries in activity diagram: For the backend-heavy process like UC006 Process Incoming Payout, it was challenging to visualize the invisible logic. Thus, we had to carefully decide how much technical detail to show in the Activity Diagram, ensuring the logic remained understandable to stakeholders while being technically accurate.

Lessons learned:

1. The power of visual abstraction: We learned that the UML models significantly reduce the ambiguity found in the written text.
2. Security by design: By explicitly modeling the misuse cases, we learned that security cannot be an afterthought. Identifying threats early in the modeling phase allowed us to design specific mitigation use cases, making the system architecture more resilient from the start.
3. Traceability is Critical: We realized that a change in one model will impact all others models. A new requirement in an Activity Diagram often requires a new attribute in the Class Diagram. This taught us the importance of traceability and iterative review to maintain a cohesive requirements model.

6. REFERENCES

1. Case Study: Etiqa Digital Payout (via eMoney) in Accelerating Customer Disbursements and Reinforcement of Financial Inclusion.
   (2025). Faculty of Computing, Universiti Teknologi Malaysia.
2. Bank Negara Malaysia. (2023).
   Risk Management in Technology (RMIT) Policy Document.
3. Wiegers, K., & Beatty, J. (2013).
   Software Requirements (3rd ed.). Microsoft Press.
4. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

7. APPENDICES

- **Appendix A : Originality/Similarity Report**

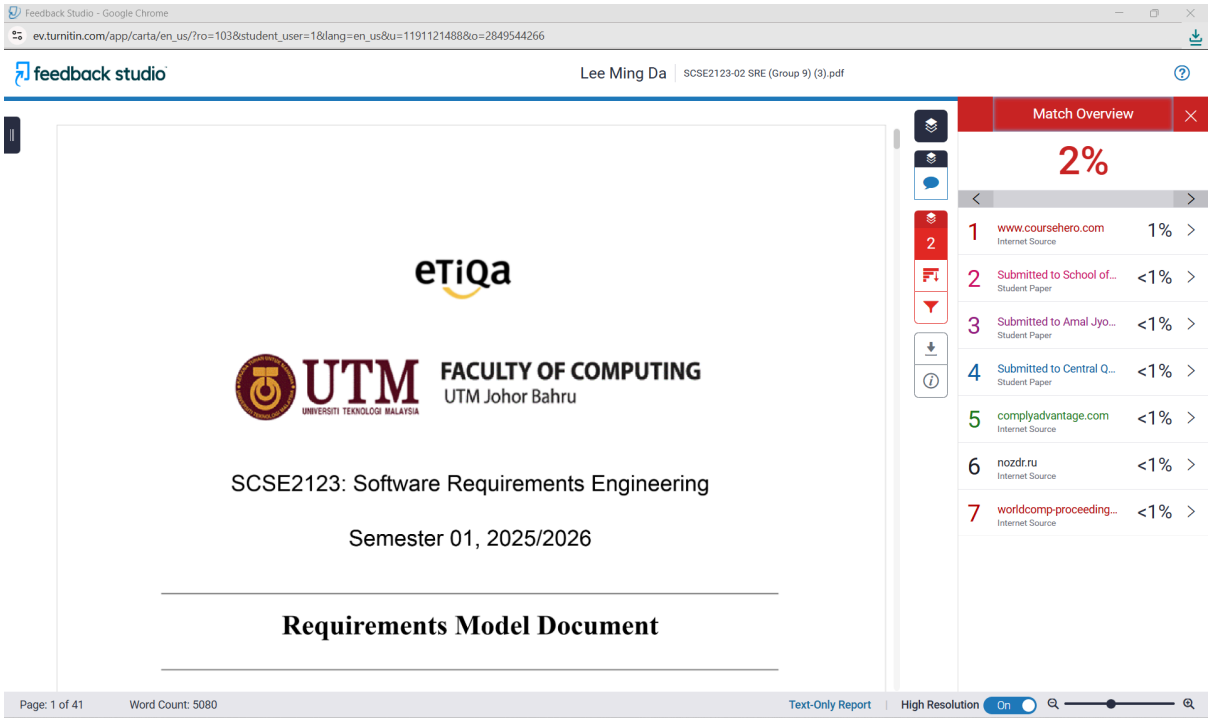


Figure 7.1: Screenshot for originality/ similarity report from Turnitin checking.

- **Appendix B : List of Prompts Used**

Prompt Used:

1. "I am writing the introduction for my 'Etiqa eMoney Payout' software requirements report. We are moving from Phase 1 (URD) to Phase 2 (Modeling). Can you suggest a logical outline for the introduction that covers the transition from text to visual models (Functional, Data, and Behavioral perspectives)?"

2. "We are defining the actors for our Use Case Diagram. We have 'Customer', 'App', and 'Source System'. We also want to model security threats. What is the best practice for modeling an 'Attacker' in UML? Should they be a primary actor, or is there a specific relationship type I should use to show they threaten specific cases?"

3. "I am designing the 'Request Payout' activity flow. I need to include a step for uploading documents. Is it better to model 'Upload Verification Documents' as a

simple action within the main flow, or should I separate it into an <<included>> use case to make the diagram cleaner? Please explain the pros and cons."

4. "We are modeling MUC02 Submit Fraudulent Claim. We want to show how the system detects this. Can you suggest how to represent a 'Mitigation' step in an Activity Diagram? Specifically, how do I visually represent the system comparing incoming data against a legacy database?"

5. "I am building the Domain Class Diagram. I have a 'PayoutRequest' class and a 'Document' class. Since a document cannot exist without a specific request, what is the correct UML relationship to use here? Aggregation or Composition? Please explain why."

6. "We faced a challenge distinguishing between the 'App' (frontend) and the 'Backend' system in our activity diagrams. Can you explain why this distinction is important in software requirements modeling? I want to include this insight in my report's conclusion."

Screenshot:

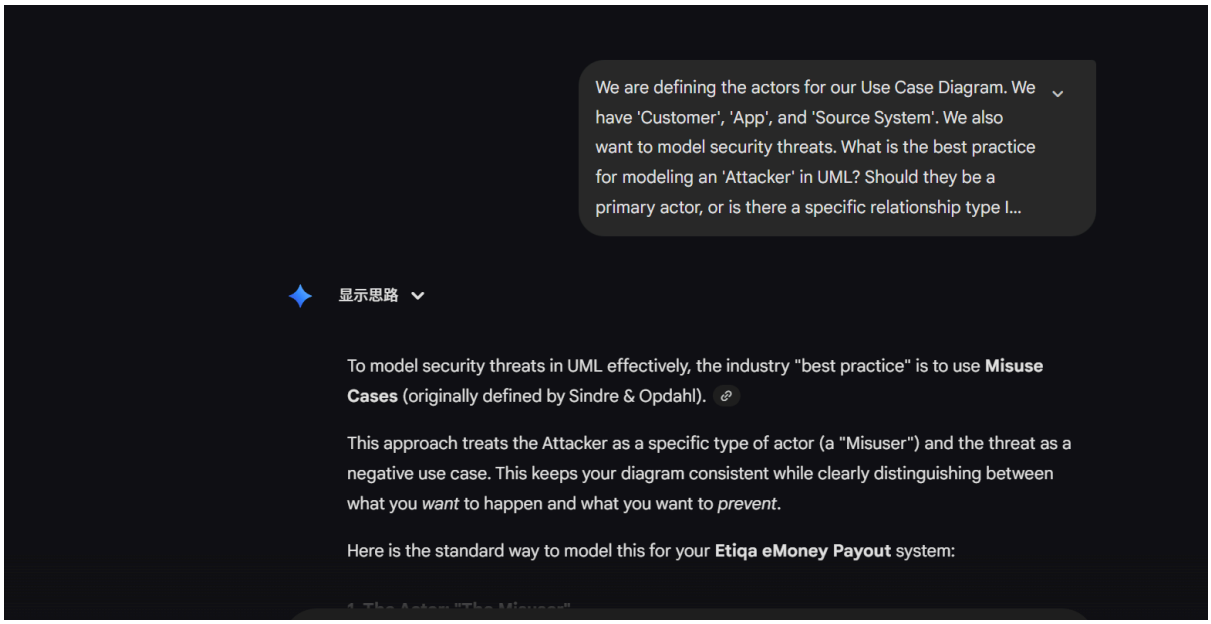


Figure 7.2: Screenshot of using prompt2

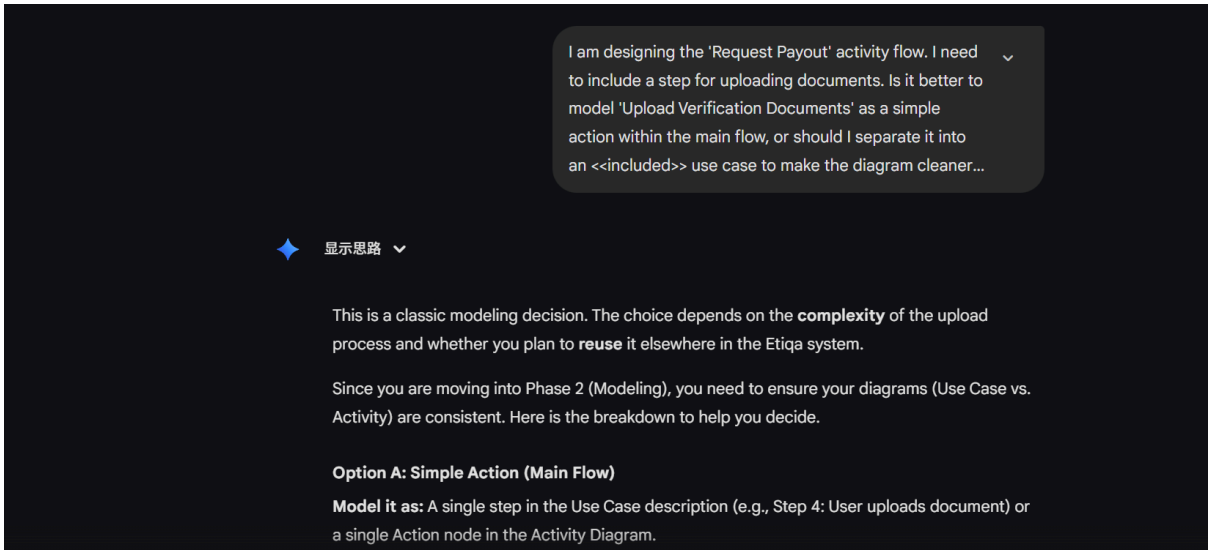


Figure 7.3: Screenshot of using prompt3

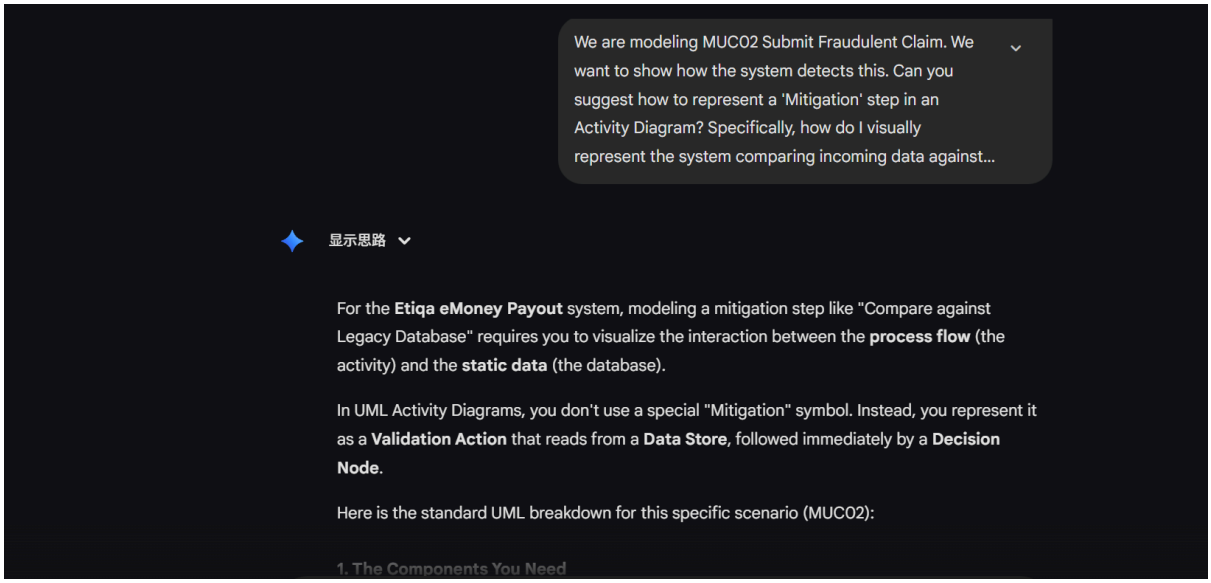


Figure 7.4: Screenshot of using prompt4

URL: https://gemini.google.com/share/407f88ff76b0

Video presentation link**:**

Phase 2 Presentation